I need to develop a project where I use object oriented paradigm and design patterns. Altough I need to use design patterns, I should only use them if it makes sense to use them in the specific context. So forcing design patterns where they don't even make sense will not get the job done.
The project can be any app, like a paint app, or a game. Programming languages supported are JavaScript, Typescript, Python, Java.
Do you think that there are opportunities to use design patterns in a way that makes sense in the following project:

```
<!DOCTYPE html>
<html lang="hu">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1.0, user-scalable=no">
  <title>Stickman Dojo Showdown 1v1 - Web UI & Backend Bridge</title>
  <script src="https://cdn.tailwindcss.com"></script>
  <script>
    tailwind.config = {
      darkMode: 'class',
      theme: {
        extend: {
          colors: {
            dojoDark: '#080c14',
            dojoPanel: '#0f172a',
            p1Blue: '#38bdf8',
            p2Red: '#f43f5e',
            neonGold: '#f59e0b'
          },
          fontFamily: {
            mono: ['Courier New', 'Consolas', 'monospace'],
            sans: ['Inter', 'system-ui', 'sans-serif']
          }
        }
      }
    }
  </script>
  <style>
    body {
      background-color: #080c14;
      color: #e2e8f0;
      user-select: none;
      -webkit-user-select: none;
      touch-action: manipulation;
    }
    .neon-blue-glow {
      box-shadow: 0 0 20px rgba(56, 189, 248, 0.3);
    }
    .neon-red-glow {
      box-shadow: 0 0 20px rgba(244, 63, 94, 0.3);
    }
    .custom-scroll::-webkit-scrollbar {
      width: 5px;
    }
    .custom-scroll::-webkit-scrollbar-track {
      background: #090d16;
    }
    .custom-scroll::-webkit-scrollbar-thumb {
      background: #1e293b;
      border-radius: 4px;
    }
    .custom-scroll::-webkit-scrollbar-thumb:hover {
      background: #334155;
```

```
    }
    kbd {
      font-family: inherit;
      box-shadow: 0 2px 0 rgba(0, 0, 0, 0.5);
    }
  </style>
</head>
<body class="min-h-screen flex flex-col items-center justify-start p-2 sm:p-5">

  <header class="w-full max-w-6xl flex flex-wrap items-center justify-between py-2.5 px-4 mb-3 border border-slate-800 bg-slate-900/80 rounded-2xl backdrop-blur-md gap-3">
    <div class="flex items-center gap-3">
      <div class="w-10 h-10 rounded-xl bg-gradient-to-br from-sky-500 to-rose-500 p-[2px] flex items-center justify-center">
        <div class="w-full h-full bg-slate-950 rounded-[10px] flex items-center justify-center text-lg">
          ⚔️
        </div>
      </div>
      <div>
        <h1 class="text-lg font-black tracking-wider font-mono text-transparent bg-clip-text bg-gradient-to-r from-sky-400 via-amber-200 to-rose-400">
          STICKMAN 1v1 ARENA
        </h1>
        <p class="text-[11px] text-slate-400">Egy billentyűzetes PvP harc + REST API Backend Bridge</p>
      </div>
    </div>

    <!-- Match series scoreboard -->
    <div class="flex items-center gap-6 font-mono text-sm bg-slate-950/80 px-4 py-1.5 rounded-xl border border-slate-800">
      <div class="flex items-center gap-2">
        <span class="w-2.5 h-2.5 rounded-full bg-sky-400 shadow-[0_0_8px_#38bdf8]"></span>
        <span class="text-sky-300 font-bold">P1 (KÉK):</span>
        <span id="p1-rounds" class="text-lg font-black text-white">0</span>
      </div>
      <span class="text-slate-600 font-bold">VS</span>
      <div class="flex items-center gap-2">
        <span id="p2-rounds" class="text-lg font-black text-white">0</span>
        <span class="text-rose-300 font-bold">:P2 (PIROS)</span>
        <span class="w-2.5 h-2.5 rounded-full bg-rose-500 shadow-[0_0_8px_#f43f5e]"></span>
      </div>
    </div>

    <!-- Audio toggle button -->
    <button id="sound-toggle-btn" class="text-xs font-mono px-3 py-1.5 rounded-lg bg-slate-800 hover:bg-slate-700 text-slate-300 border border-slate-700 transition flex items-center gap-1.5">
      <span id="sound-icon">🔊</span> Hang: <span id="sound-status" class="text-emerald-400 font-bold">BE</span>
    </button>
  </header>

  <main class="w-full max-w-6xl grid grid-cols-1 xl:grid-cols-12 gap-5 flex-1 items-start">

    <!-- BAL OLDAL: Harctér és Játékosvezérlés (8 oszlop) -->
    <section class="xl:col-span-8 flex flex-col items-center gap-3">

      <!-- HP & Energia Sávok Fejléce a pálya felett -->
      <div class="w-full grid grid-cols-2 gap-4 px-1 font-mono">
        <!-- P1 Stats -->
        <div class="flex flex-col gap-1">
          <div class="flex justify-between text-xs font-bold text-sky-400">
            <span class="flex items-center gap-1">P1 SHADOW (WASD + F/G)</span>
            <span id="p1-hp-text">100 HP</span>
          </div>
          <div class="w-full h-3.5 bg-slate-900 border border-sky-950 rounded-full overflow-hidden p-0.5 shadow-inner">
```

```
          <div id="p1-hp-bar" class="h-full bg-gradient-to-r from-sky-500 to-cyan-300 rounded-full transition-all duration-100 ease-out" style="width: 100%;"></div>
        </div>
        <!-- P1 Mana / Special Bar -->
        <div class="w-full h-1.5 bg-slate-900 border border-slate-800 rounded-full overflow-hidden">
          <div id="p1-energy-bar" class="h-full bg-amber-400 transition-all duration-75" style="width: 100%;"></div>
        </div>
      </div>

      <!-- P2 Stats -->
      <div class="flex flex-col gap-1">
        <div class="flex justify-between text-xs font-bold text-rose-400">
          <span id="p2-hp-text">100 HP</span>
          <span class="flex items-center gap-1">BLAZE (NYILAK + L/K) P2</span>
        </div>
        <div class="w-full h-3.5 bg-slate-900 border border-rose-950 rounded-full overflow-hidden p-0.5 shadow-inner flex justify-end">
          <div id="p2-hp-bar" class="h-full bg-gradient-to-l from-rose-500 to-amber-500 rounded-full transition-all duration-100 ease-out" style="width: 100%;"></div>
        </div>
        <!-- P2 Mana / Special Bar -->
        <div class="w-full h-1.5 bg-slate-900 border border-slate-800 rounded-full overflow-hidden flex justify-end">
          <div id="p2-energy-bar" class="h-full bg-amber-400 transition-all duration-75" style="width: 100%;"></div>
        </div>
      </div>

    <div class="relative w-full aspect-[16/9] max-h-[500px] rounded-2xl border border-slate-800 bg-[#070b12] overflow-hidden shadow-2xl flex items-center justify-center">
      <!-- Canvas Arena -->
      <canvas id="arena-canvas" width="800" height="450" class="w-full h-full block"></canvas>

      <!-- Kerekek / Round Vége Overlay -->
      <div id="winner-modal" class="absolute inset-0 bg-slate-950/85 backdrop-blur-md flex flex-col items-center justify-center p-6 text-center z-20 hidden transition-all">
        <div class="w-16 h-16 rounded-2xl bg-amber-500/20 border border-amber-500/40 flex items-center justify-center text-3xl mb-3 animate-bounce">
          🏆
        </div>
        <h2 id="winner-title" class="text-3xl font-black font-mono text-amber-400 tracking-wider mb-2">
          P1 GYŐZÖTT!
        </h2>
        <p id="winner-stats-summary" class="text-xs text-slate-300 font-mono mb-5 max-w-md leading-relaxed">
          Meccs időtartama: 24.2s | Bevitt sebzés: P1 100 - P2 64
        </p>

        <div class="flex flex-wrap gap-3 justify-center">
          <button id="rematch-btn" class="px-5 py-2.5 bg-gradient-to-r from-emerald-500 to-teal-500 hover:from-emerald-400 hover:to-teal-400 text-slate-950 font-black font-mono rounded-xl shadow-lg transition active:scale-95 text-sm">
            ⚔️ ÚJ MENET (SPACE)
          </button>
          <button id="post-match-btn" class="px-4 py-2.5 bg-slate-800 hover:bg-slate-700 text-sky-400 font-mono border border-sky-800/60 rounded-xl text-sm transition active:scale-95">
            📡 ADATOK BEKÜLDÉSE BACKENDRE
          </button>
        </div>
      </div>
    </div>

    <div class="w-full grid grid-cols-1 sm:grid-cols-2 gap-3 mt-1">
      <!-- P1 Controls Badge -->
```

```
        <div class="p-3 bg-slate-900/90 border border-sky-900/40 rounded-xl flex flex-col gap-1.5">
          <div class="text-xs font-bold text-sky-400 font-mono flex items-center justify-between">
            <span>🟦 JÁTÉKOS 1 (KÉK)</span>
            <span class="text-[10px] text-slate-400">Bal oldali kontroller</span>
          </div>
          <div class="text-[11px] text-slate-300 font-mono flex flex-wrap gap-1.5 items-center">
            <span>Mozgás:</span>
            <kbd class="px-1.5 py-0.5 bg-slate-800 border border-slate-700 rounded text-sky-300">W</kbd>
(Ugrás),
            <kbd class="px-1.5 py-0.5 bg-slate-800 border border-slate-700 rounded text-sky-
300">A</kbd>/<kbd class="px-1.5 py-0.5 bg-slate-800 border border-slate-700 rounded text-sky-
300">D</kbd> (Lépés),
            <kbd class="px-1.5 py-0.5 bg-slate-800 border border-slate-700 rounded text-sky-300">S</kbd>
(Blokk)
          </div>
          <div class="text-[11px] text-slate-300 font-mono flex flex-wrap gap-1.5 items-center">
            <span>Akciók:</span>
            <kbd class="px-1.5 py-0.5 bg-sky-950 border border-sky-700 text-sky-300 rounded font-
bold">F</kbd> Kardvágás
            <span class="text-slate-600">•</span>
            <kbd class="px-1.5 py-0.5 bg-amber-950 border border-amber-700 text-amber-300 rounded font-
bold">G</kbd> Energiagolyó
          </div>
        </div>

        <!-- P2 Controls Badge -->
        <div class="p-3 bg-slate-900/90 border border-rose-900/40 rounded-xl flex flex-col gap-1.5">
          <div class="text-xs font-bold text-rose-400 font-mono flex items-center justify-between">
            <span>🟥 JÁTÉKOS 2 (PIROS)</span>
            <span class="text-[10px] text-slate-400">Jobb oldali kontroller</span>
          </div>
          <div class="text-[11px] text-slate-300 font-mono flex flex-wrap gap-1.5 items-center">
            <span>Mozgás:</span>
            <kbd class="px-1.5 py-0.5 bg-slate-800 border border-slate-700 rounded text-rose-300">↑</kbd>
(Ugrás),
            <kbd class="px-1.5 py-0.5 bg-slate-800 border border-slate-700 rounded text-rose-
300">←</kbd>/<kbd class="px-1.5 py-0.5 bg-slate-800 border border-slate-700 rounded text-rose-
300">→</kbd> (Lépés),
            <kbd class="px-1.5 py-0.5 bg-slate-800 border border-slate-700 rounded text-rose-300">↓</kbd>
(Blokk)
          </div>
          <div class="text-[11px] text-slate-300 font-mono flex flex-wrap gap-1.5 items-center">
            <span>Akciók:</span>
            <kbd class="px-1.5 py-0.5 bg-rose-950 border border-rose-700 text-rose-300 rounded font-
bold">L</kbd> Kardvágás
            <span class="text-slate-600">•</span>
            <kbd class="px-1.5 py-0.5 bg-amber-950 border border-amber-700 text-amber-300 rounded font-
bold">K</kbd> Energiagolyó
          </div>
        </div>
      </div>
    </section>

    <section class="xl:col-span-4 flex flex-col gap-4">

      <!-- Meccs Statisztikai Kártya -->
      <div class="bg-slate-900 border border-slate-800 rounded-2xl p-4 flex flex-col">
        <div class="flex items-center justify-between pb-3 border-b border-slate-800 mb-3">
          <div class="flex items-center gap-2 font-mono text-xs font-bold text-slate-200">
            <span>📊 </span> AKTUÁLIS MENET STATISZTIKA
          </div>
          <span id="match-timer" class="text-xs font-mono font-bold text-amber-400">00:00.0</span>
        </div>

        <div class="grid grid-cols-2 gap-3 text-xs font-mono">
          <div class="p-2.5 rounded-xl bg-slate-950/70 border border-sky-950">
```

```
        <span class="text-sky-400 font-bold block mb-1">P1 Kék Harcos</span>
        <div class="text-slate-400 text-[11px] space-y-1">
          <div>Kiosztott: <span id="p1-dmg-dealt" class="text-white font-bold">0</span> dmg</div>
          <div>Sikeres találat: <span id="p1-hits" class="text-white font-bold">0</span></div>
          <div>Kivédett: <span id="p1-blocks" class="text-white font-bold">0</span></div>
        </div>
      </div>

      <div class="p-2.5 rounded-xl bg-slate-950/70 border border-rose-950">
        <span class="text-rose-400 font-bold block mb-1">P2 Piros Harcos</span>
        <div class="text-slate-400 text-[11px] space-y-1">
          <div>Kiosztott: <span id="p2-dmg-dealt" class="text-white font-bold">0</span> dmg</div>
          <div>Sikeres találat: <span id="p2-hits" class="text-white font-bold">0</span></div>
          <div>Kivédett: <span id="p2-blocks" class="text-white font-bold">0</span></div>
        </div>
      </div>
    </div>
  </div>

  <!-- Backend REST Inspector / Hálózati Napló -->
  <div class="bg-slate-900 border border-slate-800 rounded-2xl p-4 flex-1 flex flex-col">
    <div class="flex items-center justify-between pb-3 border-b border-slate-800 mb-2">
      <div class="flex items-center gap-2 font-mono text-xs font-bold text-slate-200">
        <span class="w-2.5 h-2.5 rounded-full bg-emerald-400 animate-ping"></span>
        REST API BRIDGE
      </div>
      <span class="text-[10px] font-mono text-slate-400">Spring Boot / FastAPI</span>
    </div>

    <p class="text-[11px] text-slate-400 mb-2 leading-relaxed">
      Ez a konzol mutatja a meccs végi JSON payloadot, amit a Python (FastAPI/Flask) vagy Java (Spring Boot) backend <code class="text-sky-300 font-mono">POST /api/matches</code> végpontja fog feldolgozni.
    </p>

    <!-- API napló ablak -->
    <div id="backend-log" class="bg-[#060911] border border-slate-800 rounded-xl p-3 font-mono text-[11px] text-slate-300 overflow-y-auto custom-scroll min-h-[220px] max-h-[300px] space-y-2">
      <!-- Naplóelemek ide renderelődnek -->
    </div>

    <div class="mt-3 pt-2 border-t border-slate-800 flex justify-between items-center text-[10px] text-slate-500 font-mono">
      <span>Státusz: <span class="text-emerald-400 font-bold">Kiszolgáló aktív</span></span>
      <button id="clear-api-log-btn" class="hover:text-slate-300 underline">Napló ürítése</button>
    </div>
  </div>

</section>
</main>

<script>
  const SoundEffects = {
    ctx: null,
    enabled: true,

    init() {
      if (!this.ctx) {
        const AudioContext = window.AudioContext || window.webkitAudioContext;
        this.ctx = new AudioContext();
      }
    },

    ensureContext() {
      this.init();
      if (this.ctx && this.ctx.state === 'suspended') {
```

```
    this.ctx.resume();
  }
},

playSlash() {
  if (!this.enabled) return;
  this.ensureContext();
  const osc = this.ctx.createOscillator();
  const gain = this.ctx.createGain();
  osc.type = 'triangle';
  osc.frequency.setValueAtTime(320, this.ctx.currentTime);
  osc.frequency.exponentialRampToValueAtTime(80, this.ctx.currentTime + 0.12);
  gain.gain.setValueAtTime(0.3, this.ctx.currentTime);
  gain.gain.exponentialRampToValueAtTime(0.01, this.ctx.currentTime + 0.12);
  osc.connect(gain);
  gain.connect(this.ctx.destination);
  osc.start();
  osc.stop(this.ctx.currentTime + 0.12);
},

playHit() {
  if (!this.enabled) return;
  this.ensureContext();
  const osc = this.ctx.createOscillator();
  const gain = this.ctx.createGain();
  osc.type = 'sawtooth';
  osc.frequency.setValueAtTime(140, this.ctx.currentTime);
  osc.frequency.exponentialRampToValueAtTime(40, this.ctx.currentTime + 0.18);
  gain.gain.setValueAtTime(0.5, this.ctx.currentTime);
  gain.gain.exponentialRampToValueAtTime(0.01, this.ctx.currentTime + 0.18);
  osc.connect(gain);
  gain.connect(this.ctx.destination);
  osc.start();
  osc.stop(this.ctx.currentTime + 0.18);
},

playBlock() {
  if (!this.enabled) return;
  this.ensureContext();
  const osc = this.ctx.createOscillator();
  const gain = this.ctx.createGain();
  osc.type = 'sine';
  osc.frequency.setValueAtTime(800, this.ctx.currentTime);
  osc.frequency.exponentialRampToValueAtTime(500, this.ctx.currentTime + 0.08);
  gain.gain.setValueAtTime(0.35, this.ctx.currentTime);
  gain.gain.exponentialRampToValueAtTime(0.01, this.ctx.currentTime + 0.08);
  osc.connect(gain);
  gain.connect(this.ctx.destination);
  osc.start();
  osc.stop(this.ctx.currentTime + 0.08);
},

playShoot() {
  if (!this.enabled) return;
  this.ensureContext();
  const osc = this.ctx.createOscillator();
  const gain = this.ctx.createGain();
  osc.type = 'sine';
  osc.frequency.setValueAtTime(600, this.ctx.currentTime);
  osc.frequency.exponentialRampToValueAtTime(1200, this.ctx.currentTime + 0.15);
  gain.gain.setValueAtTime(0.25, this.ctx.currentTime);
  gain.gain.exponentialRampToValueAtTime(0.01, this.ctx.currentTime + 0.15);
  osc.connect(gain);
  gain.connect(this.ctx.destination);
  osc.start();
  osc.stop(this.ctx.currentTime + 0.15);
```

```
  },

  playVictory() {
    if (!this.enabled) return;
    this.ensureContext();
    const notes = [261.63, 329.63, 392.00, 523.25];
    notes.forEach((freq, idx) => {
      const osc = this.ctx.createOscillator();
      const gain = this.ctx.createGain();
      osc.type = 'triangle';
      osc.frequency.setValueAtTime(freq, this.ctx.currentTime + idx * 0.1);
      gain.gain.setValueAtTime(0.3, this.ctx.currentTime + idx * 0.1);
      gain.gain.exponentialRampToValueAtTime(0.01, this.ctx.currentTime + idx * 0.1 + 0.3);
      osc.connect(gain);
      gain.connect(this.ctx.destination);
      osc.start(this.ctx.currentTime + idx * 0.1);
      osc.stop(this.ctx.currentTime + idx * 0.1 + 0.35);
    });
  }
};

const canvas = document.getElementById('arena-canvas');
const ctx = canvas.getContext('2d');

// Platformok a pályán (Talaj + 2 lebegő sziget)
const platforms = [
  { x: 0, y: 390, width: 800, height: 60, type: 'ground' },
  { x: 120, y: 280, width: 170, height: 16, type: 'floating' },
  { x: 510, y: 280, width: 170, height: 16, type: 'floating' },
  { x: 320, y: 190, width: 160, height: 16, type: 'floating' }
];

const GRAVITY = 0.65;
let particles = [];
let projectiles = [];

// Billentyűzet állapot tároló
const keys = {};

window.addEventListener('keydown', (e) => {
  keys[e.key.toLowerCase()] = true;
  keys[e.key] = true;
  if (['ArrowUp', 'ArrowDown', 'ArrowLeft', 'ArrowRight', ' '].includes(e.key)) {
    e.preventDefault();
  }
  if (e.key === ' ' && isMatchOver) {
    startNewRound();
  }
});

window.addEventListener('keyup', (e) => {
  keys[e.key.toLowerCase()] = false;
  keys[e.key] = false;
});

class Fighter {
  constructor(id, name, color, startX, facing) {
    this.id = id;
    this.name = name;
    this.color = color;
    this.startX = startX;
    this.facing = facing; // 1: jobb, -1: bal
    this.reset();
  }

  reset() {
```

```
    this.x = this.startX;
    this.y = 300;
    this.vx = 0;
    this.vy = 0;
    this.width = 36;
    this.height = 70;
    this.hp = 100;
    this.energy = 100;
    this.isGrounded = false;
    this.isAttacking = false;
    this.attackTimer = 0;
    this.attackType = null; // 'slash'
    this.isBlocking = false;
    this.hitFlash = 0;
    this.walkCycle = 0;
    this.facing = this.id === 1 ? 1 : -1;
  }

  update(enemy) {
    // Energiavisszatöltés másodpercenként
    if (this.energy < 100) {
      this.energy = Math.min(100, this.energy + 0.25);
    }

    if (this.hitFlash > 0) this.hitFlash--;

    // Támadási időzítő kezelése
    if (this.isAttacking) {
      this.attackTimer++;
      if (this.attackTimer > 16) {
        this.isAttacking = false;
        this.attackTimer = 0;
        this.attackType = null;
      }
    }

    // Gravitáció
    this.vy += GRAVITY;
    this.x += this.vx;
    this.y += this.vy;

    // Talaj- és platform-ütközés detektálás
    this.isGrounded = false;
    const fighterBottom = this.y + this.height;
    const prevBottom = fighterBottom - this.vy;

    for (const plat of platforms) {
      const withinX = this.x + this.width > plat.x && this.x < plat.x + plat.width;
      if (withinX) {
        // Fentről érkezik a platform tetejére
        if (prevBottom <= plat.y + 12 && fighterBottom >= plat.y) {
          this.y = plat.y - this.height;
          this.vy = 0;
          this.isGrounded = true;
          break;
        }
      }
    }

    // Falak (Canvas szélei)
    if (this.x < 10) this.x = 10;
    if (this.x + this.width > canvas.width - 10) this.x = canvas.width - 10 - this.width;

    // Séta fázis számláló animációhoz
    if (Math.abs(this.vx) > 0.5 && this.isGrounded) {
      this.walkCycle += 0.25;
```

```
    } else {
      this.walkCycle = 0;
    }
  }

  jump() {
    if (this.isGrounded && !this.isBlocking) {
      this.vy = -13.5;
      this.isGrounded = false;
      createDustParticles(this.x + this.width / 2, this.y + this.height);
    }
  }

  slash(enemy) {
    if (this.isAttacking || this.isBlocking) return;
    this.isAttacking = true;
    this.attackType = 'slash';
    this.attackTimer = 0;
    SoundEffects.playSlash();

    // Hitbox számítás kardcsapáshoz
    const swordRange = 55;
    const hitBoxX = this.facing === 1 ? this.x + this.width : this.x - swordRange;
    const hitBoxY = this.y + 10;
    const hitBoxW = swordRange;
    const hitBoxH = 45;

    // Ütközés az ellenféllel
    const enemyBox = { x: enemy.x, y: enemy.y, width: enemy.width, height: enemy.height };
    if (checkAABB({ x: hitBoxX, y: hitBoxY, width: hitBoxW, height: hitBoxH }, enemyBox)) {
      enemy.takeDamage(12, this);
    }
  }

  shootSpecial(enemy) {
    if (this.isAttacking || this.isBlocking || this.energy < 30) return;
    this.energy -= 30;
    SoundEffects.playShoot();

    projectiles.push({
      ownerId: this.id,
      x: this.facing === 1 ? this.x + this.width + 10 : this.x - 10,
      y: this.y + 25,
      vx: this.facing * 9,
      color: this.color,
      radius: 7,
      life: 80
    });
  }

  takeDamage(amount, attacker) {
    let finalDamage = amount;
    if (this.isBlocking) {
      // Ha az ellenfél felé fordul blokkolás közben, 80% sebzéscsökkentés
      if ((attacker.x > this.x && this.facing === 1) || (attacker.x < this.x && this.facing === -1)) {
        finalDamage = Math.round(amount * 0.2);
        SoundEffects.playBlock();
        createSparks(this.x + (this.facing === 1 ? this.width : 0), this.y + 25, '#38bdf8', 6);
        if (this.id === 1) matchStats.p1Blocks++;
        else matchStats.p2Blocks++;
      }
    }

    if (finalDamage > 0) {
      this.hp = Math.max(0, this.hp - finalDamage);
      this.hitFlash = 10;
```

```
    this.vx = (attacker.x < this.x ? 1 : -1) * 4.5; // Visszalökődés (knockback)
    SoundEffects.playHit();
    createSparks(this.x + this.width / 2, this.y + 25, this.color, 12);

    // Statisztikák regisztrálása
    if (attacker.id === 1) {
      matchStats.p1DamageDealt += finalDamage;
      matchStats.p1Hits++;
    } else {
      matchStats.p2DamageDealt += finalDamage;
      matchStats.p2Hits++;
    }

    updateStatsUI();

    if (this.hp <= 0) {
      endMatch(attacker.id);
    }
  }
}

draw() {
  ctx.save();
  ctx.lineWidth = 3.5;
  ctx.lineCap = 'round';
  ctx.lineJoin = 'round';

  // Villogás sebződéskor
  const drawColor = this.hitFlash > 0 ? '#ffffff' : this.color;
  ctx.strokeStyle = drawColor;
  ctx.fillStyle = drawColor;

  const headRadius = 10;
  const centerX = this.x + this.width / 2;
  const headY = this.y + headRadius + 4;
  const neckY = headY + headRadius;
  const pelvisY = this.y + 44;
  const footY = this.y + this.height;

  // 1. Fej rajzolása
  ctx.beginPath();
  ctx.arc(centerX, headY, headRadius, 0, Math.PI * 2);
  ctx.stroke();

  // Szemcsík a nézési irány jelzésére
  ctx.beginPath();
  const eyeOffset = this.facing * 4;
  ctx.arc(centerX + eyeOffset, headY - 1, 2, 0, Math.PI * 2);
  ctx.fill();

  // 2. Törzs (Gerinc)
  ctx.beginPath();
  ctx.moveTo(centerX, neckY);
  ctx.lineTo(centerX, pelvisY);
  ctx.stroke();

  // 3. Lábak animációja (Séta és ugrás dinamika)
  const walkSin = Math.sin(this.walkCycle) * 12;
  let leftFootX = centerX - 8 + (this.isGrounded ? walkSin : -5);
  let rightFootX = centerX + 8 - (this.isGrounded ? walkSin : -5);
  let leftKneeY = pelvisY + 12;
  let rightKneeY = pelvisY + 12;

  if (this.isBlocking) {
    // Guggoló védekező póz
    leftFootX = centerX - 12 * this.facing;
```

```
    rightFootX = centerX + 6 * this.facing;
  }

  // Bal láb
  ctx.beginPath();
  ctx.moveTo(centerX, pelvisY);
  ctx.lineTo(leftFootX, footY);
  ctx.stroke();

  // Jobb láb
  ctx.beginPath();
  ctx.moveTo(centerX, pelvisY);
  ctx.lineTo(rightFootX, footY);
  ctx.stroke();

  // 4. Karok és fegyver
  const shoulderY = neckY + 4;

  if (this.isBlocking) {
    // Védekező póz: karok a mellkas előtt keresztbe
    const shieldHandX = centerX + this.facing * 14;
    ctx.beginPath();
    ctx.moveTo(centerX, shoulderY);
    ctx.lineTo(shieldHandX, shoulderY + 4);
    ctx.lineTo(shieldHandX, shoulderY - 12);
    ctx.stroke();

    // Energia pajzs ív
    ctx.strokeStyle = '#38bdf8';
    ctx.beginPath();
    ctx.arc(centerX + this.facing * 18, headY + 8, 22, -Math.PI / 3, Math.PI / 3, this.facing === -1);
    ctx.stroke();
  } else if (this.isAttacking && this.attackType === 'slash') {
    // Dinamikus kardvágás animáció fázis szerint
    const progress = this.attackTimer / 16; // 0 .. 1
    const startAngle = this.facing === 1 ? -Math.PI * 0.75 : Math.PI * 1.75;
    const endAngle = this.facing === 1 ? Math.PI * 0.25 : Math.PI * 0.75;
    const currentAngle = startAngle + (endAngle - startAngle) * progress;

    const armLength = 18;
    const handX = centerX + Math.cos(currentAngle) * armLength;
    const handY = shoulderY + Math.sin(currentAngle) * armLength;

    // Kar rajzolása
    ctx.beginPath();
    ctx.moveTo(centerX, shoulderY);
    ctx.lineTo(handX, handY);
    ctx.stroke();

    // Kard rajzolása
    const swordLength = 40;
    const tipX = handX + Math.cos(currentAngle) * swordLength;
    const tipY = handY + Math.sin(currentAngle) * swordLength;

    ctx.strokeStyle = '#f8fafc';
    ctx.lineWidth = 4;
    ctx.beginPath();
    ctx.moveTo(handX, handY);
    ctx.lineTo(tipX, tipY);
    ctx.stroke();

    // Suhintási fényív (Slash trail)
    ctx.strokeStyle = this.color;
    ctx.lineWidth = 2;
    ctx.beginPath();
    ctx.arc(centerX, shoulderY, swordLength + 10, currentAngle - 0.5, currentAngle + 0.2);
```

```
      ctx.stroke();
    } else {
      // Alapállapot: kard a kézben lefelé tartva
      const handX = centerX + this.facing * 12;
      const handY = shoulderY + 12;

      ctx.beginPath();
      ctx.moveTo(centerX, shoulderY);
      ctx.lineTo(handX, handY);
      ctx.stroke();

      // Kard hüvelyben / kézben
      ctx.strokeStyle = '#94a3b8';
      ctx.lineWidth = 3;
      ctx.beginPath();
      ctx.moveTo(handX, handY);
      ctx.lineTo(handX + this.facing * 8, handY + 24);
      ctx.stroke();
    }

    ctx.restore();
  }
}

function checkAABB(rect1, rect2) {
  return (
    rect1.x < rect2.x + rect2.width &&
    rect1.x + rect1.width > rect2.x &&
    rect1.y < rect2.y + rect2.height &&
    rect1.y + rect1.height > rect2.y
  );
}

function createSparks(x, y, color, count = 8) {
  for (let i = 0; i < count; i++) {
    particles.push({
      x: x,
      y: y,
      vx: (Math.random() - 0.5) * 8,
      vy: (Math.random() - 0.5) * 8,
      size: Math.random() * 3 + 2,
      color: color,
      alpha: 1,
      decay: Math.random() * 0.04 + 0.03
    });
  }
}

function createDustParticles(x, y) {
  for (let i = 0; i < 5; i++) {
    particles.push({
      x: x + (Math.random() - 0.5) * 16,
      y: y,
      vx: (Math.random() - 0.5) * 2,
      vy: -Math.random() * 1.5,
      size: 3,
      color: '#475569',
      alpha: 0.6,
      decay: 0.05
    });
  }
}

let p1 = new Fighter(1, "Player 1 (Shadow)", "#38bdf8", 160, 1);
let p2 = new Fighter(2, "Player 2 (Blaze)", "#f43f5e", 600, -1);
```

```
let roundScores = { p1: 0, p2: 0 };
let isMatchOver = false;
let matchStartTime = Date.now();
let matchElapsedSeconds = 0;

const matchStats = {
  p1DamageDealt: 0,
  p1Hits: 0,
  p1Blocks: 0,
  p2DamageDealt: 0,
  p2Hits: 0,
  p2Blocks: 0
};

// UI elemek
const p1HpBar = document.getElementById('p1-hp-bar');
const p2HpBar = document.getElementById('p2-hp-bar');
const p1HpText = document.getElementById('p1-hp-text');
const p2HpText = document.getElementById('p2-hp-text');
const p1EnergyBar = document.getElementById('p1-energy-bar');
const p2EnergyBar = document.getElementById('p2-energy-bar');
const p1RoundsEl = document.getElementById('p1-rounds');
const p2RoundsEl = document.getElementById('p2-rounds');
const matchTimerEl = document.getElementById('match-timer');
const winnerModal = document.getElementById('winner-modal');
const winnerTitle = document.getElementById('winner-title');
const winnerStatsSummary = document.getElementById('winner-stats-summary');
const rematchBtn = document.getElementById('rematch-btn');
const postMatchBtn = document.getElementById('post-match-btn');
const backendLog = document.getElementById('backend-log');
const clearApiLogBtn = document.getElementById('clear-api-log-btn');
const soundToggleBtn = document.getElementById('sound-toggle-btn');
const soundStatusEl = document.getElementById('sound-status');
const soundIconEl = document.getElementById('sound-icon');

// UI Statisztikák frissítése
function updateStatsUI() {
  p1HpBar.style.width = `${Math.max(0, p1.hp)}%`;
  p2HpBar.style.width = `${Math.max(0, p2.hp)}%`;
  p1HpText.innerText = `${Math.ceil(p1.hp)} HP`;
  p2HpText.innerText = `${Math.ceil(p2.hp)} HP`;

  p1EnergyBar.style.width = `${p1.energy}%`;
  p2EnergyBar.style.width = `${p2.energy}%`;

  document.getElementById('p1-dmg-dealt').innerText = matchStats.p1DamageDealt;
  document.getElementById('p1-hits').innerText = matchStats.p1Hits;
  document.getElementById('p1-blocks').innerText = matchStats.p1Blocks;

  document.getElementById('p2-dmg-dealt').innerText = matchStats.p2DamageDealt;
  document.getElementById('p2-hits').innerText = matchStats.p2Hits;
  document.getElementById('p2-blocks').innerText = matchStats.p2Blocks;
}

function logBackendApiCall(method, endpoint, payload, responseData) {
  const entry = document.createElement('div');
  entry.className = "p-2 rounded bg-slate-900 border border-slate-800 transition-all";
  const time = new Date().toLocaleTimeString();

  entry.innerHTML = `
    <div class="flex items-center justify-between mb-1">
      <div class="flex items-center gap-1.5">
        <span class="px-1.5 py-0.5 rounded text-[10px] font-bold ${method === 'POST' ? 'bg-amber-950 text-amber-400 border border-amber-800' : 'bg-blue-950 text-blue-400 border border-blue-800'}">${method}</span>
        <span class="text-slate-300 font-bold">${endpoint}</span>
```

```
          </div>
          <span class="text-[10px] text-emerald-400 font-bold">201 CREATED</span>
        </div>
        <div class="text-[10px] text-slate-400"><b>Payload:</b> <pre class="text-sky-300 whitespace-pre-wrap break-all mt-0.5">${JSON.stringify(payload, null, 2)}</pre></div>
        <div class="text-[9px] text-slate-500 mt-1 flex justify-between">
          <span>Server: Mock Spring/FastAPI</span>
          <span>${time}</span>
        </div>
      `;
      backendLog.prepend(entry);
    }

    function sendMatchDataToBackend(winnerId) {
      const payload = {
        matchId: "match_" + Date.now(),
        timestamp: new Date().toISOString(),
        durationSeconds: parseFloat(matchElapsedSeconds.toFixed(1)),
        winner: winnerId === 1 ? "Player 1 (Shadow)" : "Player 2 (Blaze)",
        players: {
          p1: {
            name: "Player 1",
            color: "blue",
            damageDealt: matchStats.p1DamageDealt,
            hitsLanded: matchStats.p1Hits,
            blocksExecuted: matchStats.p1Blocks,
            remainingHp: p1.hp
          },
          p2: {
            name: "Player 2",
            color: "red",
            damageDealt: matchStats.p2DamageDealt,
            hitsLanded: matchStats.p2Hits,
            blocksExecuted: matchStats.p2Blocks,
            remainingHp: p2.hp
          }
        },
        currentSeriesScore: {
          p1Wins: roundScores.p1,
          p2Wins: roundScores.p2
        }
      };

      logBackendApiCall("POST", "/api/v1/matches", payload, { success: true, savedId: payload.matchId });
    }

    function endMatch(winnerId) {
      if (isMatchOver) return;
      isMatchOver = true;
      SoundEffects.playVictory();

      if (winnerId === 1) {
        roundScores.p1++;
        p1RoundsEl.innerText = roundScores.p1;
        winnerTitle.innerText = "P1 (KÉK HARCOS) GYŐZÖTT!";
        winnerTitle.className = "text-2xl sm:text-3xl font-black font-mono text-sky-400 tracking-wider mb-2";
      } else {
        roundScores.p2++;
        p2RoundsEl.innerText = roundScores.p2;
        winnerTitle.innerText = "P2 (PIROS HARCOS) GYŐZÖTT!";
        winnerTitle.className = "text-2xl sm:text-3xl font-black font-mono text-rose-500 tracking-wider mb-2";
      }

      winnerStatsSummary.innerText = `Menet idő: ${matchElapsedSeconds.toFixed(1)}s | Kiosztott sebzés: P1: ${matchStats.p1DamageDealt} HP - P2: ${matchStats.p2DamageDealt} HP`;
      winnerModal.classList.remove('hidden');
```

```
  // Automatikus szimulált API beküldés
  sendMatchDataToBackend(winnerId);
}

function startNewRound() {
  isMatchOver = false;
  winnerModal.classList.add('hidden');
  matchStartTime = Date.now();

  // Statisztikák nullázása az új menethez
  matchStats.p1DamageDealt = 0;
  matchStats.p1Hits = 0;
  matchStats.p1Blocks = 0;
  matchStats.p2DamageDealt = 0;
  matchStats.p2Hits = 0;
  matchStats.p2Blocks = 0;

  particles = [];
  projectiles = [];
  p1.reset();
  p2.reset();
  updateStatsUI();
}

function handlePlayerControls() {
  if (isMatchOver) return;

  // ================= P1 (Kék) Kontroller =================
  p1.vx = 0;
  p1.isBlocking = false;

  if (keys['s'] || keys['S']) {
    p1.isBlocking = true;
  } else {
    if (keys['a'] || keys['A']) {
      p1.vx = -4;
      p1.facing = -1;
    }
    if (keys['d'] || keys['D']) {
      p1.vx = 4;
      p1.facing = 1;
    }
    if (keys['w'] || keys['W']) {
      p1.jump();
    }
  }

  if (keys['f'] || keys['F']) {
    p1.slash(p2);
  }
  if (keys['g'] || keys['G']) {
    p1.shootSpecial(p2);
    keys['g'] = false;
    keys['G'] = false; // Egyszeri elsütés
  }

  // ================= P2 (Piros) Kontroller =================
  p2.vx = 0;
  p2.isBlocking = false;

  if (keys['ArrowDown']) {
    p2.isBlocking = true;
  } else {
    if (keys['ArrowLeft']) {
      p2.vx = -4;
```

```
      p2.facing = -1;
    }
    if (keys['ArrowRight']) {
      p2.vx = 4;
      p2.facing = 1;
    }
    if (keys['ArrowUp']) {
      p2.jump();
    }
  }

  if (keys['l'] || keys['L']) {
    p2.slash(p1);
  }
  if (keys['k'] || keys['K']) {
    p2.shootSpecial(p1);
    keys['k'] = false;
    keys['K'] = false; // Egyszeri elsütés
  }
}

function drawArenaEnvironment() {
  // Háttér gradiens
  const bgGrad = ctx.createLinearGradient(0, 0, 0, canvas.height);
  bgGrad.addColorStop(0, '#060913');
  bgGrad.addColorStop(1, '#0d1527');
  ctx.fillStyle = bgGrad;
  ctx.fillRect(0, 0, canvas.width, canvas.height);

  // Háttér dojo hold / neon fény
  ctx.save();
  const moonGrad = ctx.createRadialGradient(canvas.width / 2, 130, 10, canvas.width / 2, 130, 90);
  moonGrad.addColorStop(0, 'rgba(245, 158, 11, 0.25)');
  moonGrad.addColorStop(1, 'rgba(245, 158, 11, 0)');
  ctx.fillStyle = moonGrad;
  ctx.beginPath();
  ctx.arc(canvas.width / 2, 130, 90, 0, Math.PI * 2);
  ctx.fill();

  // Háttér díszítő dojo rácsok
  ctx.strokeStyle = 'rgba(30, 41, 59, 0.4)';
  ctx.lineWidth = 1;
  for (let x = 60; x < canvas.width; x += 80) {
    ctx.beginPath();
    ctx.moveTo(x, 40);
    ctx.lineTo(x, 390);
    ctx.stroke();
  }
  ctx.restore();

  // Platformok kirajzolása
  for (const plat of platforms) {
    if (plat.type === 'ground') {
      // Fő talaj
      ctx.fillStyle = '#0f172a';
      ctx.fillRect(plat.x, plat.y, plat.width, plat.height);

      // Talaj él neon vonala
      ctx.strokeStyle = '#334155';
      ctx.lineWidth = 3;
      ctx.beginPath();
      ctx.moveTo(plat.x, plat.y);
      ctx.lineTo(plat.x + plat.width, plat.y);
      ctx.stroke();

      // Díszítő dojo padló minta
```

```
      ctx.strokeStyle = '#1e293b';
      ctx.lineWidth = 1;
      for (let gx = 0; gx < canvas.width; gx += 40) {
        ctx.beginPath();
        ctx.moveTo(gx, plat.y);
        ctx.lineTo(gx, canvas.height);
        ctx.stroke();
      }
    } else {
      // Lebegő sziget platformok
      ctx.fillStyle = '#1e293b';
      ctx.beginPath();
      ctx.roundRect(plat.x, plat.y, plat.width, plat.height, 6);
      ctx.fill();

      // Neon fény a lebegő emelet tetején
      ctx.strokeStyle = '#38bdf8';
      ctx.lineWidth = 2;
      ctx.beginPath();
      ctx.moveTo(plat.x + 4, plat.y);
      ctx.lineTo(plat.x + plat.width - 4, plat.y);
      ctx.stroke();
    }
  }
}

function updateEntities() {
  // Lövedékek mozgatása és becsapódása
  for (let i = projectiles.length - 1; i >= 0; i--) {
    const p = projectiles[i];
    p.x += p.vx;
    p.life--;

    // Rajzolás
    ctx.save();
    ctx.fillStyle = p.color;
    ctx.shadowColor = p.color;
    ctx.shadowBlur = 10;
    ctx.beginPath();
    ctx.arc(p.x, p.y, p.radius, 0, Math.PI * 2);
    ctx.fill();
    ctx.restore();

    // Ütközés ellenféllel
    const target = p.ownerId === 1 ? p2 : p1;
    const targetBox = { x: target.x, y: target.y, width: target.width, height: target.height };
    const pBox = { x: p.x - p.radius, y: p.y - p.radius, width: p.radius * 2, height: p.radius * 2 };

    if (checkAABB(pBox, targetBox)) {
      target.takeDamage(16, p.ownerId === 1 ? p1 : p2);
      projectiles.splice(i, 1);
      continue;
    }

    // Élettartam vége vagy fal ütközés
    if (p.life <= 0 || p.x < 0 || p.x > canvas.width) {
      createSparks(p.x, p.y, p.color, 4);
      projectiles.splice(i, 1);
    }
  }

  // Részecskék (Sparks & Dust)
  for (let i = particles.length - 1; i >= 0; i--) {
    const pt = particles[i];
    pt.x += pt.vx;
    pt.y += pt.vy;
```

```
      pt.alpha -= pt.decay;

      if (pt.alpha <= 0) {
        particles.splice(i, 1);
        continue;
      }

      ctx.save();
      ctx.globalAlpha = pt.alpha;
      ctx.fillStyle = pt.color;
      ctx.beginPath();
      ctx.arc(pt.x, pt.y, pt.size, 0, Math.PI * 2);
      ctx.fill();
      ctx.restore();
    }
  }

  function gameLoop() {
    // Bemenetek kezelése
    handlePlayerControls();

    // Entitások frissítése
    p1.update(p2);
    p2.update(p1);

    // Meccs időzítő frissítése
    if (!isMatchOver) {
      matchElapsedSeconds = (Date.now() - matchStartTime) / 1000;
      const mins = Math.floor(matchElapsedSeconds / 60).toString().padStart(2, '0');
      const secs = (matchElapsedSeconds % 60).toFixed(1).padStart(4, '0');
      matchTimerEl.innerText = `${mins}:${secs}`;
    }

    // UI frissítés
    updateStatsUI();

    // Rajzolás
    ctx.clearRect(0, 0, canvas.width, canvas.height);
    drawArenaEnvironment();
    updateEntities();
    p1.draw();
    p2.draw();

    requestAnimationFrame(gameLoop);
  }

  rematchBtn.addEventListener('click', startNewRound);
  postMatchBtn.addEventListener('click', () => {
    sendMatchDataToBackend(p1.hp > 0 ? 1 : 2);
  });

  clearApiLogBtn.addEventListener('click', () => {
    backendLog.innerHTML = '<div class="text-slate-600 italic">Napló törölve.</div>';
  });

  soundToggleBtn.addEventListener('click', () => {
    SoundEffects.enabled = !SoundEffects.enabled;
    soundStatusEl.innerText = SoundEffects.enabled ? "BE" : "KI";
    soundStatusEl.className = SoundEffects.enabled ? "text-emerald-400 font-bold" : "text-rose-400 font-
bold";
    soundIconEl.innerText = SoundEffects.enabled ? "🔊 " : "🔈 ";
    if (SoundEffects.enabled) SoundEffects.ensureContext();
  });

  // Kezdeti indítás
  startNewRound();
```

```
    requestAnimationFrame(gameLoop);
  </script>
</body>
</html>
```

Design Pattern Opportunities in Stickman Dojo

Yes, this project has **genuine, natural opportunities** for several design patterns. Let me break down which ones make sense and why.

Patterns That Fit Naturally

1. State Pattern ✅ *Excellent fit*

The `Fighter` class already has implicit states that affect behavior differently:

```
IDLE → ATTACKING → BLOCKING → HIT_STUN → DEAD
```

Currently this is managed with scattered booleans (`isAttacking` , `isBlocking` , `hitFlash`), which leads to tangled conditional logic. The State pattern cleans this up meaningfully.

Why it makes sense: A fighter's `update()` , `draw()` , and response to inputs all behave fundamentally differently per state. Adding new states (e.g., `DASHING` , `STUNNED`) without State pattern means touching many existing methods.

```
interface FighterState {
    enter(fighter: Fighter): void;
    update(fighter: Fighter, enemy: Fighter): void;
    draw(fighter: Fighter, ctx: CanvasRenderingContext2D): void;
    handleAttack(fighter: Fighter, enemy: Fighter): void;
}

class IdleState implements FighterState {
    enter(fighter: Fighter) {
        fighter.vx = 0;
    }

    update(fighter: Fighter, enemy: Fighter) {
        fighter.vy += GRAVITY;
        fighter.applyMovement();
        fighter.checkPlatformCollisions();
        fighter.updateWalkCycle();
    }

    draw(fighter: Fighter, ctx: CanvasRenderingContext2D) {
        // draws resting sword-down pose
        fighter.drawBody(ctx);
        fighter.drawRestingSword(ctx);
    }

    handleAttack(fighter: Fighter, enemy: Fighter) {
        fighter.setState(new SlashState(enemy));  // transition
    }
}

class BlockingState implements FighterState {
    enter(fighter: Fighter) {
        fighter.vx = 0;  // can't move while blocking
    }

    update(fighter: Fighter, enemy: Fighter) {
        fighter.vy += GRAVITY;
```

```
        fighter.applyMovement();
        fighter.checkPlatformCollisions();
    }

    draw(fighter: Fighter, ctx: CanvasRenderingContext2D) {
        fighter.drawBody(ctx);
        fighter.drawShieldArc(ctx);  // shield arc drawn here, not in an if-block
    }

    handleAttack(fighter: Fighter, enemy: Fighter) {
        // Can't attack while blocking - do nothing
    }
}

class SlashState implements FighterState {
    private timer: number = 0;
    private readonly DURATION = 16;
    private enemy: Fighter;

    constructor(enemy: Fighter) {
        this.enemy = enemy;
    }

    enter(fighter: Fighter) {
        this.timer = 0;
        SoundEffects.playSlash();
        this.checkHit(fighter);
    }

    update(fighter: Fighter, enemy: Fighter) {
        this.timer++;
        fighter.vy += GRAVITY;
        fighter.applyMovement();
        fighter.checkPlatformCollisions();

        if (this.timer >= this.DURATION) {
            fighter.setState(new IdleState());
        }
    }

    draw(fighter: Fighter, ctx: CanvasRenderingContext2D) {
        const progress = this.timer / this.DURATION;
        fighter.drawBody(ctx);
        fighter.drawSlashAnimation(ctx, progress);  // all slash visuals here
    }

    handleAttack(fighter: Fighter, enemy: Fighter) {
        // Already attacking, ignore
    }

    private checkHit(fighter: Fighter) {
        // hitbox calculation lives here, not scattered in Fighter
        const swordRange = 55;
        const hitBox = {
            x: fighter.facing === 1 ? fighter.x + fighter.width : fighter.x - swordRange,
            y: fighter.y + 10,
            width: swordRange,
            height: 45
        };
        if (checkAABB(hitBox, this.enemy.getBoundingBox())) {
            this.enemy.takeDamage(12, fighter);
        }
    }
}

class HitStunState implements FighterState {
    private timer: number = 0;
    private readonly DURATION = 10;

    enter(fighter: Fighter) {
        this.timer = 0;
    }

    update(fighter: Fighter, enemy: Fighter) {
        this.timer++;
        fighter.vy += GRAVITY;
        fighter.applyMovement();  // knockback vx still applies
        fighter.checkPlatformCollisions();

        if (this.timer >= this.DURATION) {
            fighter.setState(new IdleState());
```

```
        }
    }

    draw(fighter: Fighter, ctx: CanvasRenderingContext2D) {
        // flash white during hit stun
        fighter.drawBody(ctx, '#ffffff');
    }

    handleAttack(fighter: Fighter, enemy: Fighter) {
        // Can't act during hit stun
    }
}
```

---

2. Observer Pattern ✅ *Good fit*

Currently, `takeDamage()` directly calls `updateStatsUI()`, `endMatch()`, and plays sounds. This creates tight coupling: the `Fighter` class knows about UI elements and game flow.

Why it makes sense: The Fighter shouldn't know that a canvas game UI exists. Multiple things need to react to the same events (HP change, hit landed, match end).

```
type GameEventType =
    | 'DAMAGE_DEALT'
    | 'BLOCK_EXECUTED'
    | 'MATCH_ENDED'
    | 'ROUND_STARTED'
    | 'HP_CHANGED';

interface GameEvent {
    type: GameEventType;
    data: Record<string, unknown>;
}

interface GameEventListener {
    onEvent(event: GameEvent): void;
}

class EventBus {
    private static instance: EventBus;
    private listeners: Map<GameEventType, GameEventListener[]> = new Map();

    static getInstance(): EventBus {
        if (!EventBus.instance) {
            EventBus.instance = new EventBus();
        }
        return EventBus.instance;
    }

    subscribe(type: GameEventType, listener: GameEventListener): void {
        if (!this.listeners.has(type)) {
            this.listeners.set(type, []);
        }
        this.listeners.get(type)!.push(listener);
    }

    emit(event: GameEvent): void {
        this.listeners.get(event.type)?.forEach(l => l.onEvent(event));
    }
}

// Listeners are separate, decoupled classes
class UIStatsListener implements GameEventListener {
    onEvent(event: GameEvent): void {
        if (event.type === 'HP_CHANGED') {
            const { fighter, hp } = event.data as { fighter: string; hp: number };
            // update the relevant HP bar
        }
        if (event.type === 'DAMAGE_DEALT') {
            // update damage counters in the stats panel
        }
    }
}
```

```
class SoundListener implements GameEventListener {
    onEvent(event: GameEvent): void {
        if (event.type === 'DAMAGE_DEALT') SoundEffects.playHit();
        if (event.type === 'BLOCK_EXECUTED') SoundEffects.playBlock();
        if (event.type === 'MATCH_ENDED') SoundEffects.playVictory();
    }
}

class MatchStatsListener implements GameEventListener {
    private stats: MatchStats;

    constructor(stats: MatchStats) {
        this.stats = stats;
    }

    onEvent(event: GameEvent): void {
        if (event.type === 'DAMAGE_DEALT') {
            const { attackerId, damage } = event.data as { attackerId: number; damage: number }
            if (attackerId === 1) {
                this.stats.p1DamageDealt += damage;
                this.stats.p1Hits++;
            } else {
                this.stats.p2DamageDealt += damage;
                this.stats.p2Hits++;
            }
        }
    }
}

// Now Fighter.takeDamage() just emits events - no UI knowledge:
takeDamage(amount: number, attacker: Fighter): void {
    const finalDamage = this.calculateDamage(amount, attacker);

    if (finalDamage > 0) {
        this.hp = Math.max(0, this.hp - finalDamage);
        this.vx = (attacker.x < this.x ? 1 : -1) * 4.5;
        this.setState(new HitStunState());

        EventBus.getInstance().emit({
            type: 'DAMAGE_DEALT',
            data: { attackerId: attacker.id, targetId: this.id, damage: finalDamage }
        });
        EventBus.getInstance().emit({
            type: 'HP_CHANGED',
            data: { fighterId: this.id, hp: this.hp }
        });

        if (this.hp <= 0) {
            EventBus.getInstance().emit({
                type: 'MATCH_ENDED',
                data: { winnerId: attacker.id }
            });
        }
    }
}
```

---

3. Strategy Pattern ✅ *Good fit for attack types*

The game already has two attack types (slash, energy ball) and could have more. Currently these are methods on `Fighter` with repeated structure.

Why it makes sense: New attacks (dash attack, uppercut) can be added without modifying `Fighter` at all.

```
interface AttackStrategy {
    readonly energyCost: number;
    readonly cooldown: number;
    execute(attacker: Fighter, enemy: Fighter): void;
}

class SlashAttack implements AttackStrategy {
```

```
    readonly energyCost = 0;
    readonly cooldown = 16;

    execute(attacker: Fighter, enemy: Fighter): void {
        const swordRange = 55;
        const hitBox = {
            x: attacker.facing === 1 ? attacker.x + attacker.width : attacker.x - swordRange,
            y: attacker.y + 10,
            width: swordRange,
            height: 45
        };
        if (checkAABB(hitBox, enemy.getBoundingBox())) {
            enemy.takeDamage(12, attacker);
        }
    }
}

class EnergyBallAttack implements AttackStrategy {
    readonly energyCost = 30;
    readonly cooldown = 0;  // no animation lock, but energy cost limits it

    execute(attacker: Fighter, enemy: Fighter): void {
        if (attacker.energy < this.energyCost) return;
        attacker.energy -= this.energyCost;

        ProjectileManager.getInstance().spawn({
            ownerId: attacker.id,
            x: attacker.facing === 1 ? attacker.x + attacker.width + 10 : attacker.x - 10,
            y: attacker.y + 25,
            vx: attacker.facing * 9,
            color: attacker.color,
            radius: 7,
            life: 80
        });
    }
}

class DashAttack implements AttackStrategy {
    readonly energyCost = 20;
    readonly cooldown = 24;

    execute(attacker: Fighter, enemy: Fighter): void {
        // new attack type added without touching Fighter
        attacker.vx = attacker.facing * 12;
        const hitBox = {
            x: attacker.x,
            y: attacker.y,
            width: attacker.width + 20,
            height: attacker.height
        };
        if (checkAABB(hitBox, enemy.getBoundingBox())) {
            enemy.takeDamage(18, attacker);
        }
    }
}

// Fighter uses strategies, doesn't implement attacks itself
class Fighter {
    private attacks: Map<string, AttackStrategy> = new Map([
        ['primary', new SlashAttack()],
        ['special', new EnergyBallAttack()]
    ]);

    useAttack(key: string, enemy: Fighter): void {
        const strategy = this.attacks.get(key);
        if (!strategy) return;
        if (this.currentState instanceof BlockingState) return;
        if (this.currentState instanceof SlashState) return;

        strategy.execute(this, enemy);
    }
}
```

---

4. Singleton Pattern ✅ *Justified use*

`SoundEffects` and the `EventBus` are global resources that should only have one instance. The current `SoundEffects` object literal is already essentially a singleton, but making it explicit prevents accidental duplication.

```
class SoundEffects {
    private static instance: SoundEffects;
    private ctx: AudioContext | null = null;
    public enabled: boolean = true;

    private constructor() {}  // prevent direct instantiation

    static getInstance(): SoundEffects {
        if (!SoundEffects.instance) {
            SoundEffects.instance = new SoundEffects();
        }
        return SoundEffects.instance;
    }

    private ensureContext(): void {
        if (!this.ctx) {
            this.ctx = new (window.AudioContext || (window as any).webkitAudioContext)();
        }
        if (this.ctx.state === 'suspended') {
            this.ctx.resume();
        }
    }

    playSlash(): void {
        if (!this.enabled) return;
        this.ensureContext();
        // ... oscillator logic
    }
    // ...other methods
}
```

What to Avoid Forcing

Pattern	Why it doesn't fit here
Factory	Only two Fighter objects are ever created with trivial construction
Decorator	No need to wrap fighters with extra behaviors at runtime
Composite	No tree structures anywhere
Template Method	State pattern already handles the variation in `update()` / `draw()` more cleanly

---

Summary: Recommended Architecture

```
EventBus (Singleton)
    ↑ emits events
Fighter
    → has FighterState (State Pattern)
        → IdleState, BlockingState, SlashState, HitStunState
    → uses AttackStrategy (Strategy Pattern)
        → SlashAttack, EnergyBallAttack, DashAttack

Listeners (Observer Pattern, subscribed to EventBus)
    → UIStatsListener     (updates HP bars, stat panels)
    → SoundListener       (plays audio)
    → MatchStatsListener  (tracks match statistics)
    → MatchFlowListener   (handles endMatch, round transitions)
```

```
SoundEffects (Singleton)
```

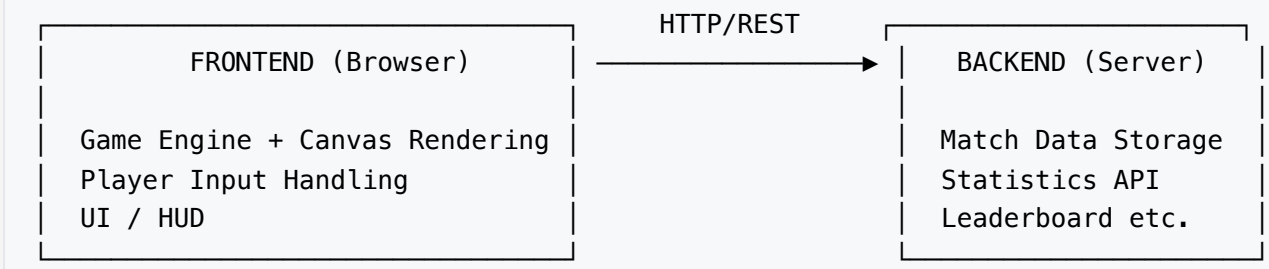
This gives you **4 patterns** used where they solve real problems: tangled state logic, tight coupling between Fighter and UI, and inflexible attack definitions.

A team of four university students will work on this project. We will develop the whole codebase from scratch. What kind of splitting of the development tasks/features would make sense? How should we distribute the tasks? Maybe along the Model-View-Controller split?

Team Task Distribution for Stickman Dojo

First: Understand What You're Actually Building

Before splitting tasks, clarify the architecture. This project has two distinct parts:



MVC makes sense, but needs to be applied **per layer**, not globally across both.

Recommended Split: 4 Roles

Person	Responsibility
A	Game Logic / Model (Fighter, Physics, Projectiles)
B	Rendering / View (Canvas drawing, Particles, HUD)
C	Input & Game Flow / Controller (Keys, Round management)
D	Backend (REST API, persistence, data model)

Person A — Game Logic (Model Layer)

Core responsibility: Everything that *computes* game state. No drawing, no DOM, no HTTP.

Owns these classes:

```
// The pure game state – no rendering knowledge whatsoever
class Fighter {
    id: number;
    name: string;
    x: number; y: number;
    vx: number; vy: number;
    hp: number; energy: number;
    facing: number;
    isGrounded: boolean;
    currentState: FighterState;   // → State Pattern (Person A designs this)
```

```
    update(enemy: Fighter): void { }
    takeDamage(amount: number, attacker: Fighter): void { }
    getBoundingBox(): Rectangle { }
    reset(): void { }
}

// State pattern — Person A owns all states
interface FighterState {
    enter(fighter: Fighter): void;
    update(fighter: Fighter, enemy: Fighter): void;
    handleAttackInput(fighter: Fighter, enemy: Fighter): void;
}

class IdleState implements FighterState { }
class BlockingState implements FighterState { }
class SlashState implements FighterState { }
class HitStunState implements FighterState { }

// Attack strategies — Person A owns these
interface AttackStrategy {
    execute(attacker: Fighter, enemy: Fighter): void;
}

class SlashAttack implements AttackStrategy { }
class EnergyBallAttack implements AttackStrategy { }
```

```
// Physics
class PhysicsEngine {
    static applyGravity(fighter: Fighter): void { }
    static checkPlatformCollisions(fighter: Fighter, platforms: Platform[]): void { }
    static checkWallBounds(fighter: Fighter, canvasWidth: number): void { }
}

// Projectiles (data only, not drawing)
class Projectile {
    x: number; y: number;
    vx: number;
    ownerId: number;
    radius: number;
    life: number;
    color: string;

    update(): void { }
    isExpired(): boolean { }
    getBoundingBox(): Rectangle { }
}

class ProjectileManager {
    private projectiles: Projectile[] = [];

    spawn(config: ProjectileConfig): void { }
    update(fighters: Fighter[]): void { }   // moves + checks hits
    getAll(): Projectile[] { }              // Person B reads this for drawing
}
```

```
// Match statistics (pure data)
class MatchStats {
    p1DamageDealt: number = 0;
    p1Hits: number = 0;
    p1Blocks: number = 0;
    p2DamageDealt: number = 0;
    p2Hits: number = 0;
    p2Blocks: number = 0;

    reset(): void { }
    recordHit(attackerId: number, damage: number): void { }
    recordBlock(defenderId: number): void { }
    toPayload(): MatchPayload { }   // Person D uses this shape
}

// Platform data
class Platform {
    x: number; y: number;
    width: number; height: number;
    type: 'ground' | 'floating';
}
```

Communicates with others via: EventBus (emits events, never imports UI or renderer)

Person B — Rendering (View Layer)

Core responsibility: Everything that *draws* to the canvas or updates the DOM. Reads game state, never modifies it.

Owns these classes:

```
// Draws a fighter given its current state — no game logic
class FighterRenderer {
    draw(fighter: Fighter, ctx: CanvasRenderingContext2D): void {
        // Reads fighter.currentState to decide which visual to show
        // But does NOT change fighter.currentState
    }

    private drawBody(fighter: Fighter, ctx: CanvasRenderingContext2D, color: string): void { }
    private drawSlashAnimation(fighter: Fighter, ctx: CanvasRenderingContext2D, progress: numbe
    private drawBlockPose(fighter: Fighter, ctx: CanvasRenderingContext2D): void { }
    private drawRestingPose(fighter: Fighter, ctx: CanvasRenderingContext2D): void { }
}

// Draws the arena background, platforms, moon glow etc.
class ArenaRenderer {
    draw(ctx: CanvasRenderingContext2D, platforms: Platform[]): void { }

    private drawBackground(ctx: CanvasRenderingContext2D): void { }
    private drawGround(ctx: CanvasRenderingContext2D, platform: Platform): void { }
    private drawFloatingPlatform(ctx: CanvasRenderingContext2D, platform: Platform): void { }
}

// Particle system — owned entirely by Person B
class ParticleSystem {
    private particles: Particle[] = [];

    spawnSparks(x: number, y: number, color: string, count: number): void { }
    spawnDust(x: number, y: number): void { }
    update(): void { }
    draw(ctx: CanvasRenderingContext2D): void { }
}

// Draws projectiles (reads from ProjectileManager)
class ProjectileRenderer {
    draw(projectiles: Projectile[], ctx: CanvasRenderingContext2D): void { }
}

// Updates DOM elements (HP bars, energy bars, timer, stats panel)
class HUDRenderer {
    private elements: HUDElements;   // cached DOM references

    updateHP(fighter: Fighter): void { }
    updateEnergy(fighter: Fighter): void { }
    updateTimer(elapsed: number): void { }
    updateStats(stats: MatchStats): void { }
    updateRoundScore(scores: RoundScores): void { }
}

// Winner modal, round transition overlays
class ModalView {
    showWinner(winnerId: number, stats: MatchStats, elapsed: number): void { }
    hide(): void { }
}
```

Communicates with others via:

- Reads `Fighter`, `Projectile[]`, `Platform[]`, `MatchStats` (read-only)
- Listens to EventBus for `HP_CHANGED`, `MATCH_ENDED`, `DAMAGE_DEALT`

Person C — Input & Game Flow (Controller Layer)

Core responsibility: Translates raw keyboard events into game actions. Manages round lifecycle.

Owns these classes:

```
// Captures and stores keyboard state
class InputManager {
    private keys: Map<string, boolean> = new Map();

    constructor() {
        window.addEventListener('keydown', (e) => this.onKeyDown(e));
        window.addEventListener('keyup',   (e) => this.onKeyUp(e));
    }

    private onKeyDown(e: KeyboardEvent): void { }
    private onKeyUp(e: KeyboardEvent): void { }

    isDown(key: string): boolean { }
    consume(key: string): void { }   // for one-shot keys like shooting
}

// Maps input to fighter actions — one per fighter
class PlayerController {
    private fighter: Fighter;
    private bindings: KeyBindings;
    private input: InputManager;

    constructor(fighter: Fighter, bindings: KeyBindings, input: InputManager) { }

    // Called every frame by the game loop
    processInput(enemy: Fighter): void {
        if (this.input.isDown(this.bindings.block)) {
            // tell fighter to block
        }
        if (this.input.isDown(this.bindings.left)) {
            // tell fighter to move left
        }
        // etc.
    }
}

// Key binding configuration — easy to extend for gamepads later
interface KeyBindings {
    left: string;
    right: string;
    jump: string;
    block: string;
    primaryAttack: string;
    specialAttack: string;
}

const P1_BINDINGS: KeyBindings = {
    left: 'a', right: 'd', jump: 'w',
    block: 's', primaryAttack: 'f', specialAttack: 'g'
};

const P2_BINDINGS: KeyBindings = {
    left: 'ArrowLeft', right: 'ArrowRight', jump: 'ArrowUp',
    block: 'ArrowDown', primaryAttack: 'l', specialAttack: 'k'
};

// Manages round start/end, scores, match lifecycle
class MatchController {
    private roundScores: RoundScores = { p1: 0, p2: 0 };
    private matchStartTime: number = 0;
    private isMatchOver: boolean = false;
    private eventBus: EventBus;

    startNewRound(): void { }
```

```
    endMatch(winnerId: number): void { }
    getElapsedSeconds(): number { }
    getRoundScores(): RoundScores { }
}

// The main game loop — ties everything together
class GameLoop {
    private controllers: PlayerController[];
    private matchController: MatchController;
    private fighters: Fighter[];
    private projectileManager: ProjectileManager;
    private renderer: GameRenderer;   // Person B's top-level renderer

    start(): void {
        requestAnimationFrame(() => this.tick());
    }

    private tick(): void {
        this.controllers.forEach(c => c.processInput(...));
        this.fighters.forEach(f => f.update(...));
        this.projectileManager.update(this.fighters);
        this.renderer.render(...);
        requestAnimationFrame(() => this.tick());
    }
}
```

Communicates with others via:

- Calls methods on `Fighter` (from Person A)
- Emits events to `EventBus`
- Wires up button click listeners, delegates to `MatchController`

Person D — Backend

Core responsibility: The server. Stores match results, serves statistics.

Owns these classes (Python/Java):

```
# FastAPI example

# --- Data Models (Pydantic) ---
class PlayerMatchData(BaseModel):
    name: str
    color: str
    damageDealt: int
    hitsLanded: int
    blocksExecuted: int
    remainingHp: float

class MatchPayload(BaseModel):
    matchId: str
    timestamp: datetime
    durationSeconds: float
    winner: str
    players: dict[str, PlayerMatchData]
    currentSeriesScore: dict[str, int]

class MatchRecord(BaseModel):
    id: int
    matchId: str
    timestamp: datetime
    winner: str
    durationSeconds: float
```

```
# --- Repository (data access) ---
class MatchRepository:
    def save(self, payload: MatchPayload) -> MatchRecord: ...
    def find_all(self) -> list[MatchRecord]: ...
    def find_by_id(self, match_id: str) -> MatchRecord | None: ...
    def get_leaderboard(self) -> list[LeaderboardEntry]: ...

# --- Service (business logic) ---
class MatchService:
    def __init__(self, repo: MatchRepository): ...

    def record_match(self, payload: MatchPayload) -> MatchRecord: ...
    def get_statistics(self) -> GlobalStats: ...
    def get_leaderboard(self) -> list[LeaderboardEntry]: ...

# --- API Routes ---
@router.post("/api/v1/matches", status_code=201)
async def create_match(payload: MatchPayload): ...

@router.get("/api/v1/matches")
async def list_matches(): ...

@router.get("/api/v1/leaderboard")
async def get_leaderboard(): ...

@router.get("/api/v1/statistics")
async def get_statistics(): ...
```

Also owns on the frontend side:

```
// The HTTP bridge between game and backend
class BackendApiClient {
    private baseUrl: string;

    async postMatch(payload: MatchPayload): Promise<MatchRecord> { }
    async getLeaderboard(): Promise<LeaderboardEntry[]> { }
}

// The log panel in the UI
class ApiLogView {
    logRequest(method: string, endpoint: string, payload: unknown): void { }
    logResponse(status: number, data: unknown): void { }
    clear(): void { }
}
```

Interface Agreements (Crucial for Parallel Work)

Define these together **on day one** so everyone can work independently:

```
// Agreed shared types — committed to a shared types.ts file
interface Rectangle {
    x: number; y: number;
    width: number; height: number;
}

interface RoundScores {
    p1: number;
    p2: number;
}

// Events agreed between A, B, C
type GameEventType =
    | 'DAMAGE_DEALT'
    | 'BLOCK_EXECUTED'
    | 'MATCH_ENDED'
    | 'ROUND_STARTED'
    | 'HP_CHANGED'
    | 'PROJECTILE_SPAWNED';
```

```
// Payload shape agreed between C (frontend) and D (backend)
interface MatchPayload {
    matchId: string;
    timestamp: string;
    durationSeconds: number;
    winner: string;
    players: {
        p1: PlayerData;
        p2: PlayerData;
    };
    currentSeriesScore: RoundScores;
}
```

Timeline Suggestion

```
Week 1: Foundation
├── All: Agree on types.ts, EventBus interface, MatchPayload shape
├── A: Fighter class skeleton + PhysicsEngine
├── B: ArenaRenderer + FighterRenderer (with stub/mock fighter data)
├── C: InputManager + PlayerController wiring
└── D: Backend project setup + POST /api/v1/matches endpoint

Week 2: Core Features
├── A: State pattern (all states), AttackStrategy, ProjectileManager
├── B: ParticleSystem, HUDRenderer, ProjectileRenderer
├── C: MatchController, GameLoop, button event wiring
└── D: Database layer, GET endpoints, ApiLogView

Week 3: Integration + Polish
├── All: Wire everything together through EventBus
├── A+C: Fine-tune physics, attack balance
├── B+C: Visual feedback (hit flash, screen shake)
└── D+C: BackendApiClient, test real HTTP calls
```

Key Rules for the Team

```
┌──────────────────────────────────────────────────────────┐
│  Person A (Model) never imports from B, C, or DOM APIs   │
│  Person B (View)  never modifies game state              │
│  Person C (Ctrl)  never draws to canvas directly         │
│  Person D (Back)  owns the MatchPayload contract         │
│  Everyone communicates through EventBus or method calls  │
│  on agreed interfaces, not concrete classes              │
└──────────────────────────────────────────────────────────┘
```

This split gives each person a **coherent, testable, independently developable** piece where the design patterns land naturally within their own domain.

I have 2 questions:
1. Did I understand it correctly that communication between the Fighter and other Model classes and the UI goes through EventBus and also directly, because UI reads Fighter state in every frame. Does it even make sense to use EventBus to communicate health changes to the UI, instead of the UI just simply reading health data from Fighter just like it reads its position and state?
Or does this make the code cleaner or the execution of the program more effective if health changes are propagated through events instead of the UI directly reading health from the fighters in every frame?
2. The implementation that I sent to you did not contain a PhysicsEngine class to control the fighters positions, but I like the idea that this is implemented in a separate class. Is it a correct approach if the moveLeft, moveRight, jump methods in the Fighter class only set the horizontal and vertical speed of the Fighter object and the PhysicsEngine will set its position after calculating collisions with platforms? The PhysicsEngine would receive the fighters and the platforms in its constructor and then in every frame,

calculate where the fighter should be, based on its speed and surrounding platforms and set the fighter's position accordingly.

Answers to Both Questions

Question 1: EventBus vs Direct Reading for UI Updates

Your understanding is correct, and your instinct to question it is also correct.

The honest answer: for HP specifically, direct reading is fine

Look at what the game loop already does every frame:

```
private tick(): void {
    this.controllers.forEach(c => c.processInput(...));
    this.fighters.forEach(f => f.update(...));
    this.projectileManager.update(this.fighters);
    this.renderer.render(...);   // reads fighter.x, fighter.y, fighter.facing...
    requestAnimationFrame(() => this.tick());
}
```

The renderer already reads `fighter.x`, `fighter.y`, `fighter.facing`, `fighter.currentState` every single frame to draw the character. Reading `fighter.hp` and `fighter.energy` in the same pass is **completely consistent** with this approach. There is no meaningful difference between position and HP from the renderer's perspective — they are both just properties of the fighter's current state.

```
class HUDRenderer {
    // This is perfectly fine - consistent with how position is handled
    update(p1: Fighter, p2: Fighter, stats: MatchStats, elapsed: number): void {
        this.updateHPBar(p1);
        this.updateHPBar(p2);
        this.updateEnergyBar(p1);
        this.updateEnergyBar(p2);
        this.updateTimer(elapsed);
        this.updateStatPanel(stats);
    }
}
```

Where EventBus genuinely earns its place

The EventBus is most valuable for **one-time discrete events** that trigger reactions, not for continuous state that is already being polled:

```
Continuous state (poll every frame):    One-time events (emit once):
─────────────────────────────────────   ──────────────────────────
fighter.x, fighter.y       ✓ poll       Match ended              → emit
fighter.hp                 ✓ poll       Hit landed               → emit
fighter.energy             ✓ poll       Block executed           → emit
fighter.currentState       ✓ poll       Projectile spawned       → emit
                                        Round started            → emit
                                        Sound trigger            → emit
```

The clearest example of why EventBus matters for discrete events:

```
// WITHOUT EventBus - Fighter must know about sound, stats, and match flow
takeDamage(amount: number, attacker: Fighter): void {
    this.hp -= amount;
    SoundEffects.playHit();                    // Fighter knows about audio system
```

```
    matchStats.recordHit(attacker.id, amount); // Fighter knows about stats tracker
    createSparks(this.x, this.y, this.color);  // Fighter knows about particle system
    if (this.hp <= 0) {
        endMatch(attacker.id);                 // Fighter knows about match flow
    }
}

// WITH EventBus - Fighter knows about nothing except itself
takeDamage(amount: number, attacker: Fighter): void {
    this.hp -= amount;
    this.setState(new HitStunState());

    EventBus.emit({ type: 'DAMAGE_DEALT', data: { attackerId: attacker.id, damage: amount } });

    if (this.hp <= 0) {
        EventBus.emit({ type: 'MATCH_ENDED', data: { winnerId: attacker.id } });
    }
}
```

`SoundListener`, `MatchStatsListener`, `ParticleListener`, and `MatchController` each independently react to these events. Fighter is completely decoupled from all of them.

Summary for Question 1

```
HP bar updates every frame?  → Direct read, same as position. No EventBus needed.
Sound on hit?                → EventBus. Fighter shouldn't know audio exists.
Stats tracking on hit?       → EventBus. Fighter shouldn't know stats exist.
Match ending on death?       → EventBus. Fighter shouldn't control match flow.
Particle burst on hit?       → EventBus. Fighter shouldn't know particles exist.
```

Question 2: PhysicsEngine Setting Fighter Position

Your proposed approach is correct and clean. There is one important detail worth thinking through carefully.

Your proposed design

```
class PhysicsEngine {
    private fighters: Fighter[];
    private platforms: Platform[];

    constructor(fighters: Fighter[], platforms: Platform[]) {
        this.fighters = fighters;
        this.platforms = platforms;
    }

    update(): void {
        for (const fighter of this.fighters) {
            this.applyGravity(fighter);
            this.applyVelocity(fighter);
            this.resolvePlatformCollisions(fighter);
            this.resolveWallBounds(fighter);
        }
    }
}
```

And Fighter methods only set velocity, never position directly:

```
class Fighter {
    moveLeft(): void {
        this.vx = -4;
        this.facing = -1;
    }
```

```
    moveRight(): void {
        this.vx = 4;
        this.facing = 1;
    }

    jump(): void {
        if (this.isGrounded) {
            this.vy = -13.5;
            // does NOT set this.y - PhysicsEngine does that
        }
    }

    stopHorizontal(): void {
        this.vx = 0;
    }
}
```

This is a clean separation. Fighter expresses **intent** through velocity. PhysicsEngine resolves **outcome** by applying that velocity against the world.

The detail worth thinking about: who owns `isGrounded`?

`isGrounded` is currently set inside the collision detection logic, which now lives in PhysicsEngine. But `Fighter.jump()` reads it. This is fine as long as you are clear that PhysicsEngine **writes** `isGrounded` and Fighter only **reads** it:

```
class Fighter {
    // PhysicsEngine writes this, Fighter reads it
    isGrounded: boolean = false;

    jump(): void {
        if (this.isGrounded) {   // reads the flag PhysicsEngine maintains
            this.vy = -13.5;
        }
    }
}

class PhysicsEngine {
    private resolvePlatformCollisions(fighter: Fighter): void {
        fighter.isGrounded = false;  // reset each frame

        for (const platform of this.platforms) {
            if (this.isLandingOnTop(fighter, platform)) {
                fighter.y = platform.y - fighter.height;
                fighter.vy = 0;
                fighter.isGrounded = true;  // PhysicsEngine sets this
                break;
            }
        }
    }
}
```

This is a straightforward and honest dependency. PhysicsEngine owns the ground truth about whether the fighter is grounded, which makes sense because only the collision detection code actually knows that.

One alternative worth knowing: static methods

If you find that passing fighters in the constructor feels awkward (for example, fighters are reset between rounds but the PhysicsEngine persists), static methods are a reasonable alternative:

```
class PhysicsEngine {
    // No constructor, no state - purely a calculation tool
    static update(fighter: Fighter, platforms: Platform[]): void {
        PhysicsEngine.applyGravity(fighter);
        PhysicsEngine.applyVelocity(fighter);
        PhysicsEngine.resolvePlatformCollisions(fighter, platforms);
        PhysicsEngine.resolveWallBounds(fighter);
```

```
    }

    private static applyGravity(fighter: Fighter): void {
        fighter.vy += GRAVITY;
    }

    private static applyVelocity(fighter: Fighter): void {
        fighter.x += fighter.vx;
        fighter.y += fighter.vy;
    }

    private static resolvePlatformCollisions(fighter: Fighter, platforms: Platform[]): void {
        fighter.isGrounded = false;
        // ...collision logic
    }

    private static resolveWallBounds(fighter: Fighter): void {
        if (fighter.x < 10) fighter.x = 10;
        if (fighter.x + fighter.width > CANVAS_WIDTH - 10) {
            fighter.x = CANVAS_WIDTH - 10 - fighter.width;
        }
    }
}
```

Called from the game loop:

```
private tick(): void {
    this.controllers.forEach(c => c.processInput(...));

    // Either approach works:
    PhysicsEngine.update(p1, this.platforms);  // static version
    PhysicsEngine.update(p2, this.platforms);

    this.projectileManager.update(this.fighters);
    this.renderer.render(...);
    requestAnimationFrame(() => this.tick());
}
```

Which approach to choose

```
Constructor injection                  Static methods
──────────────────────────             ──────────────────────────
PhysicsEngine has state                PhysicsEngine is stateless
Good if physics config                 Good if physics is purely
changes at runtime (e.g.               a set of calculations with
gravity zones, slow-motion)            no configuration needed

For this game: either works, static is slightly simpler
```

For Stickman Dojo specifically, since gravity is a constant and platforms do not change, the static approach is marginally simpler. But if you later want to add a slow-motion effect or a low-gravity platform zone, the instance approach makes that easier to extend.

Írd le kérlek magyarul, de az osztályok angol neveit megtartva, Frontend-Model, Frontend-View, Frontend-Controller és Backend szerinti bontásban, hogy milyen osztályok tartoznak az adott szekcióhoz, azok szerepe nagy vonalakban mi, milyen fő metódusaik és/vagy adattagjaik vannak és ezek szerepe nagy vonalakban mi. Külön részletezd továbbá hogy hogyan fognak egymással kommunikálni ezek az osztályok, különös tekintettel az egyes szekciókon átívelő kommunikációra, mint például hogy a FighterRenderer közvetlenül a Fighter osztály állapotát olvassa, a HUDRenderer és a MatchStatsListener osztályok viszont csak közvetve, az EventBus-on keresztül.

A Frontend-View szekcióból kérlek hagyd ki a ParticleSystem és a ModalView osztályokat, mintha nem is léteznének.

Stickman Dojo — Osztályok és Kommunikáció

Frontend — Model

Ez a réteg a játék teljes belső állapotát és logikáját tartalmazza. Nem tud a canvasről, a DOM-ról, a hálózatról, vagy bármilyen megjelenítésről. Kizárólag adatokat tárol és számításokat végez.

`Fighter`

A játékos karaktert reprezentáló osztály. Tárolja a harcos teljes állapotát, és kifejezi a mozgási szándékát sebességváltoztatáson keresztül — a pozícióját azonban nem ő számolja ki, azt a `PhysicsEngine` állítja be.

Adattagok:

Adattag	Szerep
`id: number`	A harcos azonosítója (1 vagy 2)
`name: string`	A harcos neve
`color: string`	A harcos színe (megjelenítéshez olvassa a View)
`x, y: number`	Pozíció — a `PhysicsEngine` írja, mindenki más olvassa
`vx, vy: number`	Sebesség — a Fighter írja, a `PhysicsEngine` olvassa
`width, height: number`	Ütközési terület mérete
`hp: number`	Életpontok száma
`energy: number`	Energia (speciális támadáshoz)
`facing: number`	Nézési irány (1: jobb, -1: bal)
`isGrounded: boolean`	A `PhysicsEngine` írja, a `Fighter` olvassa (ugráshoz)
`currentState: FighterState`	Az aktuális állapot — a State Pattern alapja

Metódusok:

Metódus	Szerep
`moveLeft()`	Beállítja: `vx = −4` , `facing = −1`
`moveRight()`	Beállítja: `vx = 4` , `facing = 1`
`stopHorizontal()`	Beállítja: `vx = 0`
`jump()`	Ha `isGrounded` , beállítja: `vy = −13.5`
`setState(state)`	Állapotváltás, meghívja az új állapot `enter()` metódusát
`update(enemy)`	Delegálja a hívást a `currentState.update()` -re; energiavisszatöltés
`takeDamage(amount, attacker)`	Sebzést számol, HP-t csökkent, `EventBus` -on eseményt küld

Metódus	Szerep
`getBoundingBox()`	Visszaad egy `Rectangle`-t az ütközésvizsgálathoz
`reset()`	Visszaállítja a kezdőállapotot (új menethez)

`FighterState` (interfész) + állapotosztályok

A State Pattern megvalósítása. Minden állapot önállóan kezeli, hogy az adott állapotban hogyan viselkedjen a harcos és mit lehet tenni.

Az interfész:

```
interface FighterState {
    enter(fighter: Fighter): void;
    update(fighter: Fighter, enemy: Fighter): void;
    handleAttackInput(fighter: Fighter, enemy: Fighter): void;
}
```

Megvalósított állapotok:

Osztály	Mikor aktív	Különleges viselkedés
`IdleState`	Alapállapot, mozgás	Fogadja a mozgás- és támadásinputokat
`BlockingState`	S / ↓ lenyomva	`vx = 0`, blokk animációt jelez a Viewnak, támadás nem indítható
`SlashState`	F / L leütésekor	Tartja a támadás időtartamát (`attackTimer`), ütközést vizsgál, lejárta után visszavált `IdleState`-re
`HitStunState`	Sebzés érte	Rövid ideig tart (`hitFlash` csökkentése), ez alatt nem reagál inputra

---

`AttackStrategy` (interfész) + támadásosztályok

A Strategy Pattern megvalósítása. A `Fighter` nem implementálja a támadási logikát közvetlenül — helyette stratégiaobjektumokat hív meg.

Az interfész:

```
interface AttackStrategy {
    readonly energyCost: number;
    execute(attacker: Fighter, enemy: Fighter): void;
}
```

Osztály	Szerep
`SlashAttack`	Közelharci ütés: hitboxot számol, ha eltalálja az ellenfelet, meghívja `enemy.takeDamage()`-t

Osztály	Szerep
`EnergyBallAttack`	Energiagolyót hoz létre a `ProjectileManager`-en keresztül, levonja az energiaköltséget

---

`PhysicsEngine`

A fizikai számítások elvégzése: gravitáció, sebesség alkalmazása, platformokkal és falakkal való ütközés feloldása. A Fighter **nem** számítja ki saját pozícióját — ezt kizárólag a `PhysicsEngine` végzi.

Metódusok:

Metódus	Szerep
`static update(fighter, platforms)`	Egyetlen belépési pont: meghívja az alábbi metódusokat sorban
`private static applyGravity(fighter)`	`vy += GRAVITY`
`private static applyVelocity(fighter)`	`x += vx`, `y += vy`
`private static resolvePlatformCollisions(fighter, platforms)`	Megállapítja, hogy a harcos platformon áll-e; beállítja `isGrounded`-ot, korrigálja `y`-t és `vy`-t
`private static resolveWallBounds(fighter)`	Megakadályozza, hogy a harcos kilépjen a canvas szélein

`Projectile`

Egyetlen repülő lövedék adatait tárolja.

Adattagok: `x, y, vx, ownerId, radius, life, color`

Metódusok:

Metódus	Szerep
`update()`	`x += vx`, `life--`
`isExpired()`	`life <= 0` vagy canvas határán kívül van
`getBoundingBox()`	Ütközésvizsgálathoz

`ProjectileManager`

Az összes aktív lövedéket kezeli: mozgatja őket, vizsgálja az ellenféllel való ütközést, és eltávolítja a lejárt lövedékeket.

Metódusok:

Metódus	Szerep
`spawn(config)`	Új `Projectile`-t hoz létre és hozzáadja a listához
`update(fighters)`	Minden lövedéket mozgat, ütközést vizsgál, lejártakat törli; találat esetén `fighter.takeDamage()`-t hív
`getAll()`	A `ProjectileRenderer` olvassa minden képkockában

`MatchStats`

Az aktuális menet statisztikáit tárolja. Az `EventBus`-on érkező eseményekre reagálva a `MatchStatsListener` írja, a `HUDRenderer` és a `BackendApiClient` olvassa.

Adattagok: `p1DamageDealt, p1Hits, p1Blocks, p2DamageDealt, p2Hits, p2Blocks`

Metódusok:

Metódus	Szerep
`recordHit(attackerId, damage)`	A megfelelő számlálókat növeli
`recordBlock(defenderId)`	A blokk számlálót növeli
`reset()`	Nulláz mindent új menet elején
`toPayload()`	`MatchPayload` objektumot ad vissza a backendnek küldéshez

`Platform`

Egyszerű adatosztály, egy platform pozícióját és típusát írja le.

Adattagok: `x, y, width, height, type: 'ground' | 'floating'`

`EventBus`

A Singleton Pattern és az Observer Pattern kombinációja. A Model eseményeket küld rajta keresztül anélkül, hogy tudna a hallgatókról. Mindenki, aki reagálni szeretne ezekre az eseményekre, feliratkozik rá.

Metódusok:

Metódus	Szerep
`static getInstance()`	Visszaadja az egyetlen példányt
`subscribe(type, listener)`	Feliratkozás egy eseménytípusra

Metódus	Szerep
`emit(event)`	Értesíti az adott típusra feliratkozott összes hallgatót

Eseménytípusok:

Típus	Mikor kerül kiküldésre
`DAMAGE_DEALT`	`Fighter.takeDamage()` — sikeres sebzéskor
`BLOCK_EXECUTED`	`Fighter.takeDamage()` — sikeres blokkoláskor
`MATCH_ENDED`	`Fighter.takeDamage()` — HP nullára csökken
`ROUND_STARTED`	`MatchController.startNewRound()`
`PROJECTILE_SPAWNED`	`EnergyBallAttack.execute()`

Frontend — View

Ez a réteg kizárólag megjelenítéssel foglalkozik. Olvas a Model állapotából, de soha nem módosítja azt. Két úton kap információt: egyes osztályok közvetlenül olvassák a Model objektumait képkockánként, mások az `EventBus`-on keresztül értesülnek az eseményekről.

`ArenaRenderer`

A háttér és a platformok kirajzolása a canvasre.

Metódusok:

Metódus	Szerep
`draw(ctx, platforms)`	Háttérszín, hold/fényrács, majd minden platform kirajzolása
`private drawBackground(ctx)`	Gradiens háttér és díszítő rácsok
`private drawGround(ctx, platform)`	Talaj és padlóminta
`private drawFloatingPlatform(ctx, platform)`	Lebegő sziget és neon él

Kommunikáció: A `GameLoop` hívja meg képkockánként; `Platform[]` tömböt kap paraméterként.

`FighterRenderer`

Egyetlen harcos kirajzolása a canvasre. Olvassa a Fighter állapotát, de soha nem módosítja.

Metódusok:

Metódus	Szerep
`draw(fighter, ctx)`	Belépési pont: a `fighter.currentState` alapján delegál a megfelelő rajzolómetódushoz
`private drawBody(fighter, ctx, color)`	Fej, törzs kirajzolása
`private drawRestingPose(fighter, ctx)`	Alapállapot: kard lefelé tartva
`private drawSlashAnimation(fighter, ctx, progress)`	Kardvágás animáció a `SlashState.attackTimer` alapján
`private drawBlockPose(fighter, ctx)`	Védekező póz és pajzsív

Kommunikáció: Közvetlenül olvassa a `Fighter` adattagjait képkockánként (`x` , `y` , `facing` , `currentState` , `color` , stb.). Ez teljesen analóg azzal, ahogy a pozíciót olvassa — nincs szükség `EventBus` -ra, mert ez folyamatos állapotolvasás, nem egyszeri esemény.

`ProjectileRenderer`

Az összes aktív lövedék kirajzolása.

Metódusok:

Metódus	Szerep
`draw(projectiles, ctx)`	Végigiterál a lövedékeken, mindegyiket neon izzással kirajzolja

Kommunikáció: A `ProjectileManager.getAll()` által visszaadott tömböt olvassa közvetlenül, képkockánként.

`HUDRenderer`

A DOM-alapú felhasználói felület frissítése: HP-sávok, energiasávok, statisztikák, időzítő, meneteredmény.

Adattagok: Cachelelt DOM referenciák (HP-sávok, szövegmezők, stb.), hogy ne kelljen képkockánként `getElementById` -t hívni.

Metódusok:

Metódus	Szerep
`updateHP(fighter)`	HP-sáv szélességét és szövegét frissíti
`updateEnergy(fighter)`	Energiasáv szélességét frissíti
`updateTimer(elapsed)`	Az időzítő szövegét frissíti

Metódus	Szerep
`updateRoundScore(scores)`	A menetszámláló értékeit frissíti
`updateStats(stats)`	A statisztikai panelt frissíti (sebzés, találat, blokk)

Kommunikáció: A `Fighter` és `MatchStats` objektumokat közvetlenül olvassa képkockánként. Ugyanaz az elv, mint a `FighterRenderer` esetében: ezek folyamatosan változó értékek, amelyeket úgyis minden képkockában meg kell jeleníteni, ezért az `EventBus` felesleges közvetítő lenne.

`MatchStatsListener`

Feliratkozik az `EventBus`-ra, és a beérkező események alapján frissíti a `MatchStats` objektumot.

Metódusok:

Metódus	Szerep
`onEvent(event)`	`DAMAGE_DEALT` esetén `stats.recordHit()`-et hív; `BLOCK_EXECUTED` esetén `stats.recordBlock()`-ot hív; `ROUND_STARTED` esetén `stats.reset()`-et hív

Kommunikáció: Kizárólag az `EventBus`-on keresztül értesül eseményekről. Azért nem közvetlen híváson keresztül működik, mert a `Fighter` nem tudhat a statisztikai rendszer létezéséről.

`SoundListener`

Feliratkozik az `EventBus`-ra, és a beérkező események alapján hangeffektusokat játszik le.

Metódusok:

Metódus	Szerep
`onEvent(event)`	`DAMAGE_DEALT` → `SoundEffects.playHit()`; `BLOCK_EXECUTED` → `SoundEffects.playBlock()`; `MATCH_ENDED` → `SoundEffects.playVictory()`; `PROJECTILE_SPAWNED` → `SoundEffects.playShoot()`

Kommunikáció: Kizárólag az `EventBus`-on keresztül. A `Fighter` nem tudhat az audio rendszerről.

`SoundEffects`

A Singleton Pattern megvalósítása a hangrendszerre. Az egyetlen `AudioContext` példányt és az összes hangszintézis logikát tartalmazza.

Metódusok: `playSlash()`, `playHit()`, `playBlock()`, `playShoot()`, `playVictory()`, `ensureContext()`

`ApiLogView`

A backend kommunikáció naplóját jeleníti meg a DOM-ban.

Metódusok:

Metódus	Szerep
`logRequest(method, endpoint, payload)`	Új bejegyzést illeszt a naplóba
`clear()`	Törli a napló tartalmát

Kommunikáció: A `BackendApiClient` hívja közvetlenül, miután egy kérés elküldésre kerül.

Frontend — Controller

Ez a réteg köti össze a Model-t és a View-t. Kezeli a bemeneti eseményeket, irányítja a játék menetét, és minden képkockában koordinálja a frissítési sorrendet.

`InputManager`

A billentyűzet állapotát tárolja. Nem tud a harcosokról, csupán rögzíti, hogy éppen mely billentyűk vannak lenyomva.

Adattagok: `private keys: Map<string, boolean>`

Metódusok:

Metódus	Szerep
`isDown(key)`	Visszaadja, hogy az adott billentyű le van-e nyomva
`consume(key)`	Törli az adott billentyű állapotát (egyszeri elsütéshez, pl. energiagolyóhoz)

`PlayerController`

Egyetlen játékos inputját dolgozza fel és fordítja le Fighter-metódus hívásokra. Minden képkockában a `GameLoop` hívja meg.

Adattagok: `fighter: Fighter`, `bindings: KeyBindings`, `input: InputManager`

Metódusok:

Metódus	Szerep
`processInput(enemy)`	Végigvizsgálja az összes kötött billentyűt; mozgási és támadási metódusokat hív a `Fighter`-en

---

`KeyBindings` (interfész)

A billentyűkiosztást írja le. Könnyen cserélhető, így például gamepad támogatás hozzáadása nem igényli a `PlayerController` módosítását.

```
interface KeyBindings {
    left: string; right: string; jump: string;
    block: string; primaryAttack: string; specialAttack: string;
}
```

`MatchController`

A menet életciklusát kezeli: menet indítása, befejezése, pontszámok nyilvántartása.

Adattagok: `roundScores: RoundScores`, `matchStartTime: number`, `isMatchOver: boolean`

Metódusok:

Metódus	Szerep
`startNewRound()`	Reseteli a harcosokat és a statisztikákat, `ROUND_STARTED` eseményt küld az `EventBus`-ra
`onMatchEnded(event)`	`MATCH_ENDED` eseményre reagál: pontszámot növel, `HUDRenderer`-t értesít, `BackendApiClient`-et hívja
`getElapsedSeconds()`	Az aktuális menet eltelt ideje

Kommunikáció: Feliratkozik a `MATCH_ENDED` eseményre az `EventBus`-on.

`GameLoop`

A játék fő ciklusa. Minden képkockában sorban elvégzi az összes frissítési és rajzolási lépést.

Metódusok:

Metódus	Szerep
`start()`	Elindítja a `requestAnimationFrame` ciklust
`private tick()`	Egy képkocka teljes feldolgozása a helyes sorrendben

```
private tick(): void {
    // 1. Input feldolgozás
    this.p1Controller.processInput(this.p2);
    this.p2Controller.processInput(this.p1);

    // 2. Model frissítés
    PhysicsEngine.update(this.p1, this.platforms);
    PhysicsEngine.update(this.p2, this.platforms);
    this.p1.update(this.p2);
    this.p2.update(this.p1);
    this.projectileManager.update([this.p1, this.p2]);

    // 3. Időzítő frissítés
    const elapsed = this.matchController.getElapsedSeconds();

    // 4. Canvas kirajzolás
    this.arenaRenderer.draw(this.ctx, this.platforms);
    this.projectileRenderer.draw(this.projectileManager.getAll(), this.ctx);
    this.fighterRenderer.draw(this.p1, this.ctx);
    this.fighterRenderer.draw(this.p2, this.ctx);

    // 5. HUD frissítés
    this.hudRenderer.updateHP(this.p1);
    this.hudRenderer.updateHP(this.p2);
    this.hudRenderer.updateEnergy(this.p1);
    this.hudRenderer.updateEnergy(this.p2);
    this.hudRenderer.updateTimer(elapsed);
    this.hudRenderer.updateStats(this.matchStats);

    requestAnimationFrame(() => this.tick());
}
```

Backend

A backend a meccs adatait fogadja, tárolja és szolgáltatja. A frontend `BackendApiClient` osztálya kommunikál vele HTTP-n keresztül.

`MatchPayload` (adatmodell)

A frontend által küldött JSON struktúra leírása. Ez a frontend és backend közötti szerződés.

Adattagok: `matchId, timestamp, durationSeconds, winner, players: { p1, p2 }, currentSeriesScore`

`MatchRepository`

Az adatbázis-hozzáférés egyetlen helye. A `MatchService` használja.

Metódusok:

Metódus	Szerep
`save(payload)`	Elmenti a meccs adatait, visszaadja a mentett rekordot
`findAll()`	Az összes eddigi menet listája
`findById(matchId)`	Egyetlen menet lekérdezése azonosító alapján

Metódus	Szerep
`getLeaderboard()`	Összesített győzelmi statisztika

`MatchService`

Az üzleti logika rétege. A `MatchRepository`-t használja, de nem tud a HTTP rétegről.

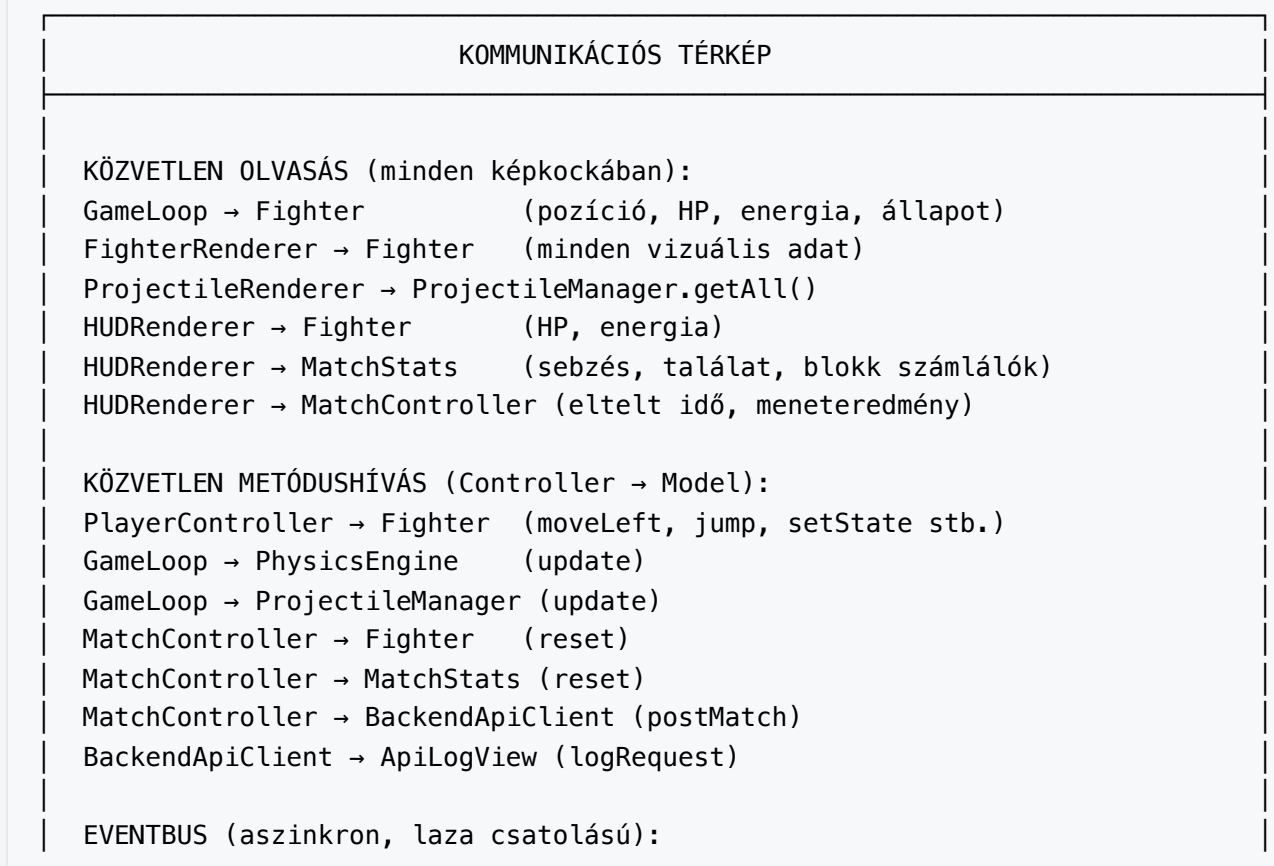
Metódusok:

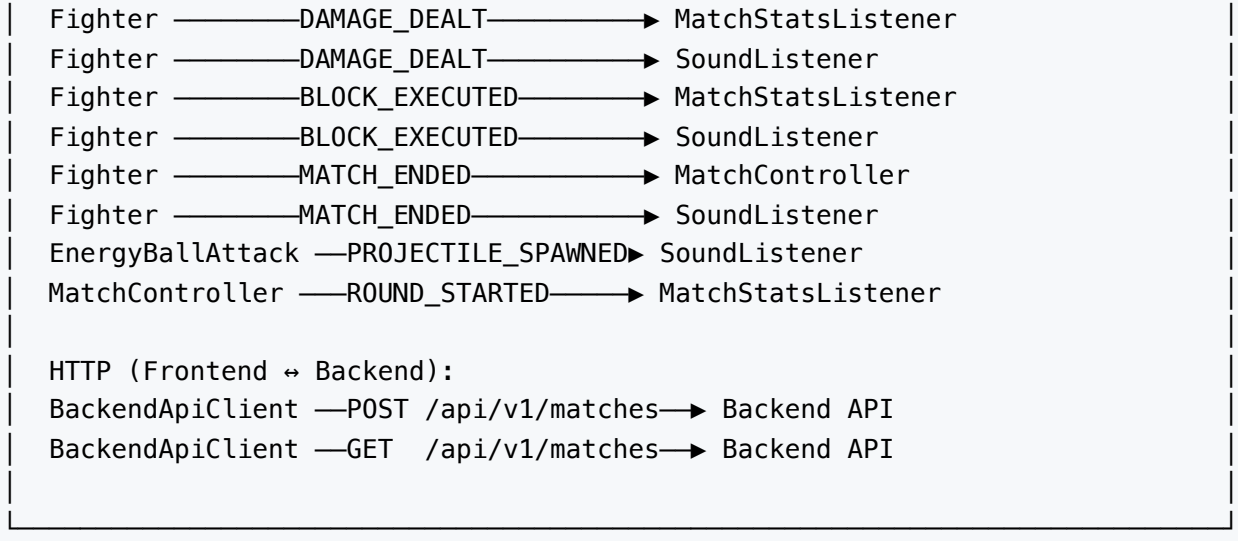
Metódus	Szerep
`recordMatch(payload)`	Validálja az adatokat, majd `repository.save()`-t hív
`getStatistics()`	Globális statisztikákat számol (pl. átlagos meccsidő)
`getLeaderboard()`	Ranglistát ad vissza

API végpontok

Végpont	Metódus	Leírás
`/api/v1/matches`	`POST`	Új menet mentése — a `MatchService.recordMatch()`-t hívja
`/api/v1/matches`	`GET`	Az összes menet listája
`/api/v1/leaderboard`	`GET`	Ranglista
`/api/v1/statistics`	`GET`	Globális statisztikák

Kommunikáció összefoglalása




Fighter ──DAMAGE_DEALT──▶ MatchStatsListener
Fighter ──DAMAGE_DEALT──▶ SoundListener
Fighter ──BLOCK_EXECUTED──▶ MatchStatsListener
Fighter ──BLOCK_EXECUTED──▶ SoundListener
Fighter ──MATCH_ENDED──▶ MatchController
Fighter ──MATCH_ENDED──▶ SoundListener
EnergyBallAttack ──PROJECTILE_SPAWNED▶ SoundListener
MatchController ──ROUND_STARTED──▶ MatchStatsListener
HTTP (Frontend ↔ Backend):
BackendApiClient ──POST /api/v1/matches──▶ Backend API
BackendApiClient ──GET /api/v1/matches──▶ Backend API